



Generative Adversarial Networks

<https://hku-data8018.github.io/>

Bo Dai

Spring 2026



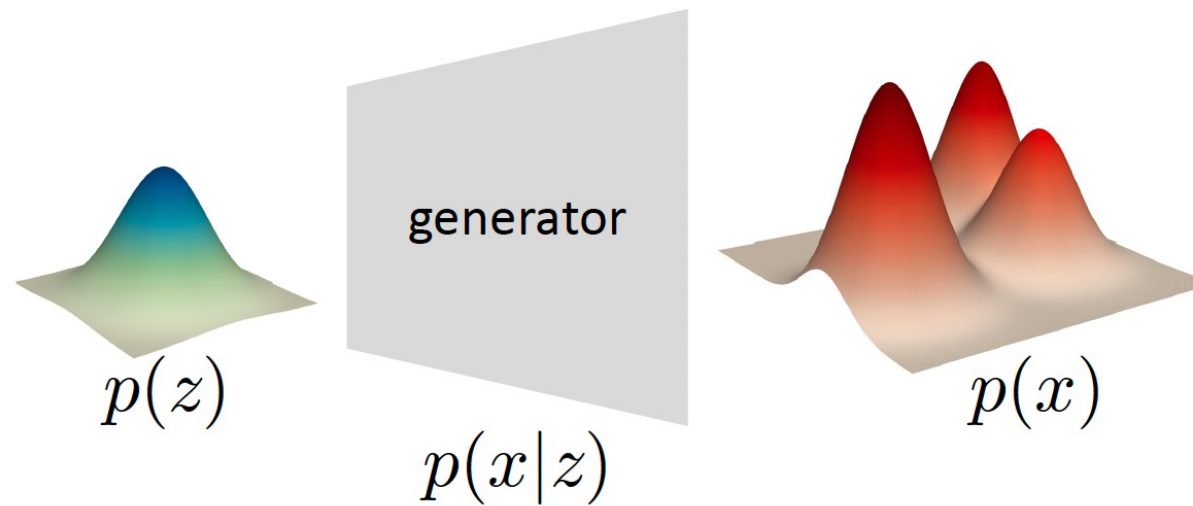
- Slides reference
 - MIT EECS Fall 2024, 6.S978 Deep Generative Models, <https://mit-6s978.github.io/>
 - Introduction to Generative Adversarial Networks, <https://github.com/DSC-UI-SRIN/Introduction-to-GAN>

Generative Adversarial Networks

- **Generative**: contrary to “Discriminative” and “Deterministic”
- **Adversarial**: a two-player game with different goals
- **Networks**: SGD + backprop for problem solving

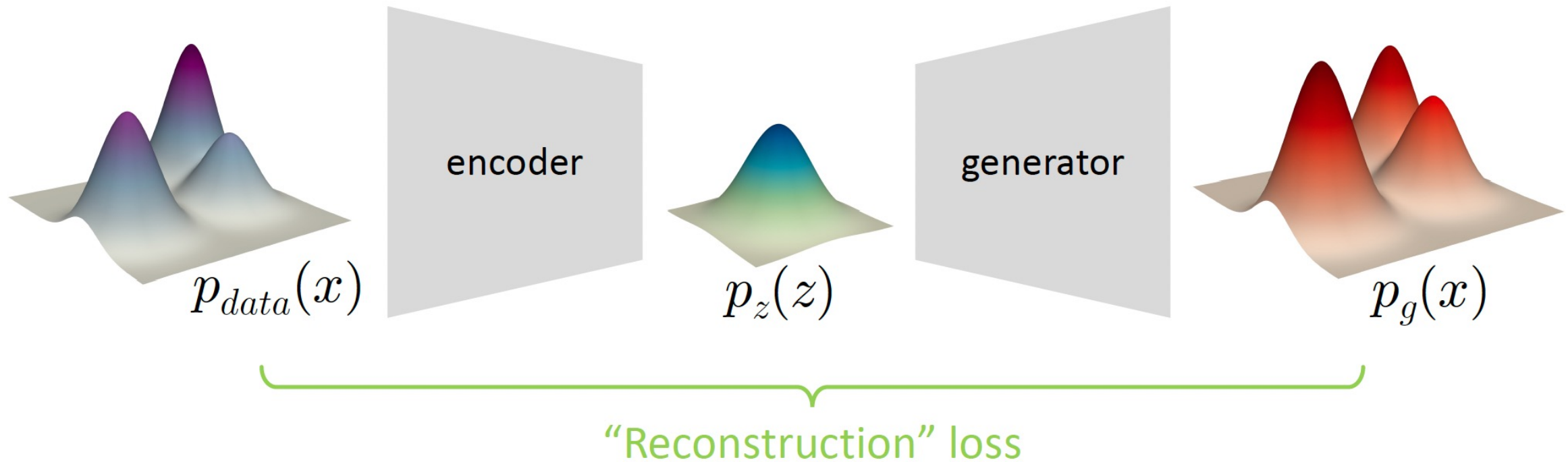
Recap: Latent Variable Models

- **z**: latent variables
- **x**: observed variables



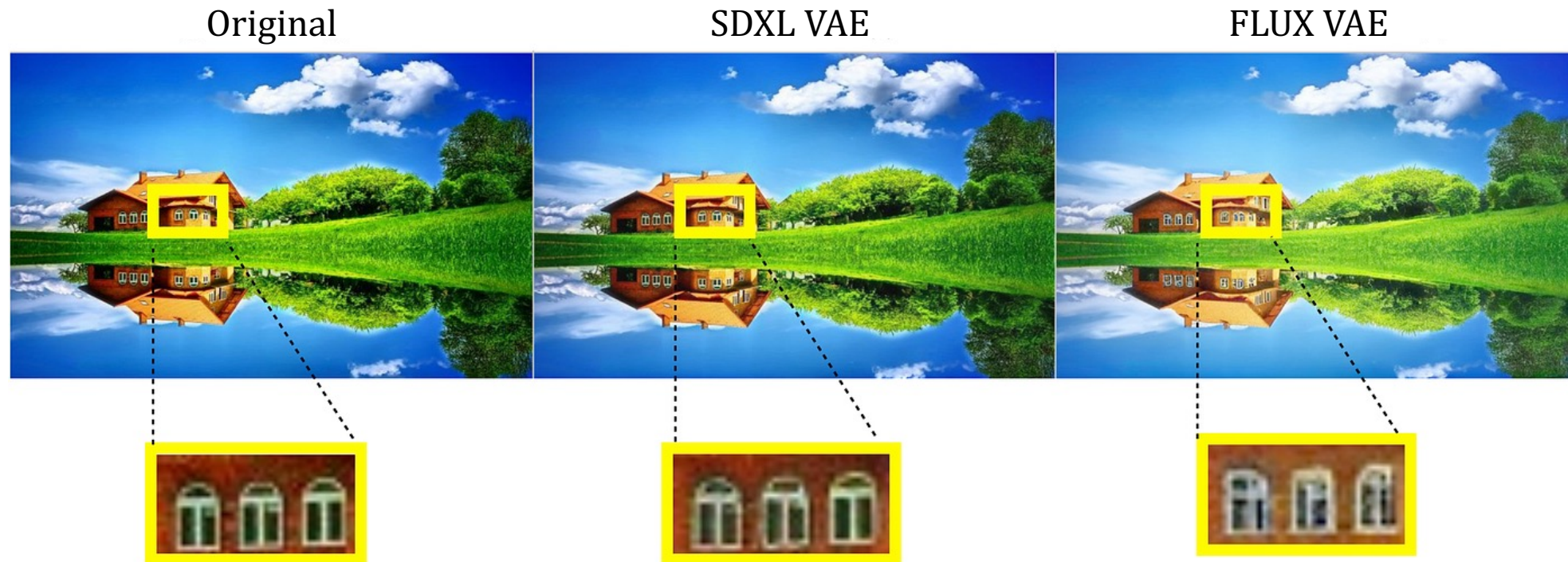
Recap: VAE

- Encoding data distribution p_{data} into latent distribution p_z



Recap: issues of VAE

- **Blurry outputs:** Compression ratio is too high; issue of the training objective; ...



Recap: issues of VAE

- Blurry outputs: Compression ratio is too high; **issue of the training objective**; ...

Implication of **the reconstruction loss**

- Elements (e.g. pixels) are **independently** distributed
- Each element follows a **simple** distribution (Gaussian / Bernoulli / ...)
- **Assumptions are too strict for high-dimensional variables**

Recap: issues of VAE

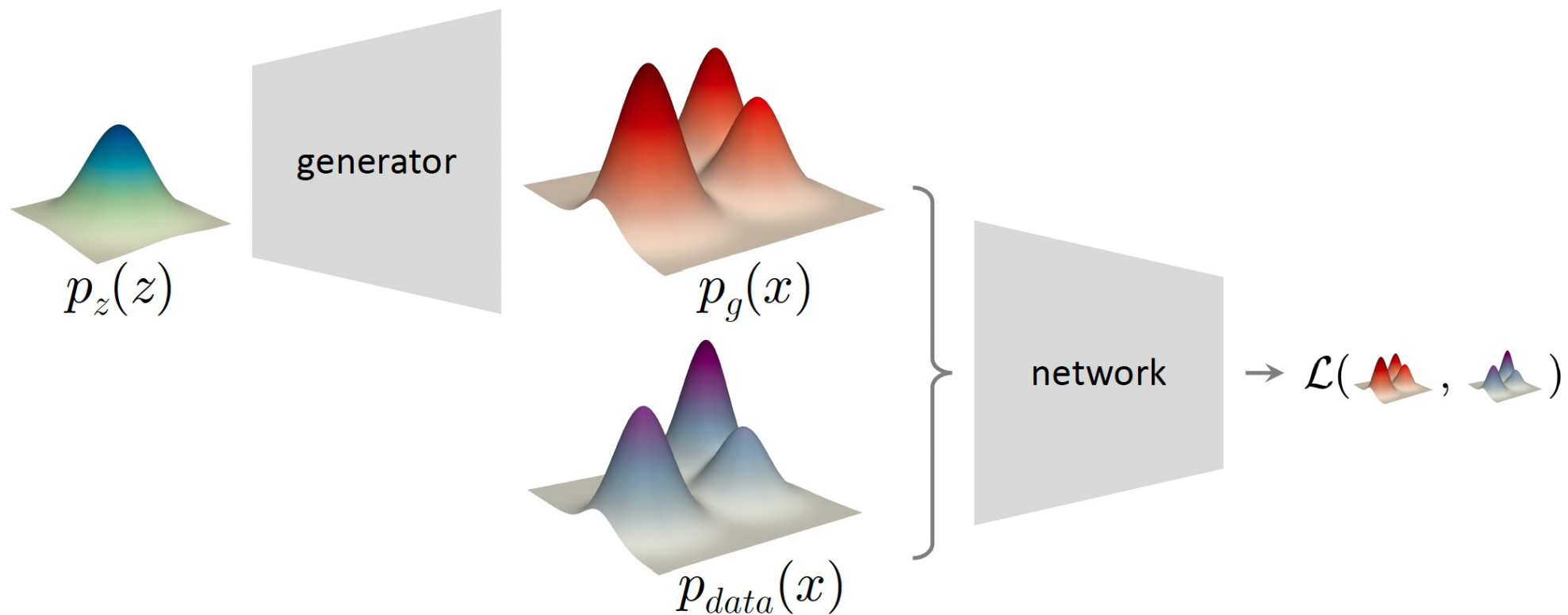
- Blurry outputs: Compression ratio is too high; issue of the training objective; ...

Implication of the reconstruction loss

- Elements (e.g. pixels) are independently distributed
- Each element follows a simple distribution (Gaussian / Bernoulli / ...)
- Assumptions are too strict for high-dimensional variables

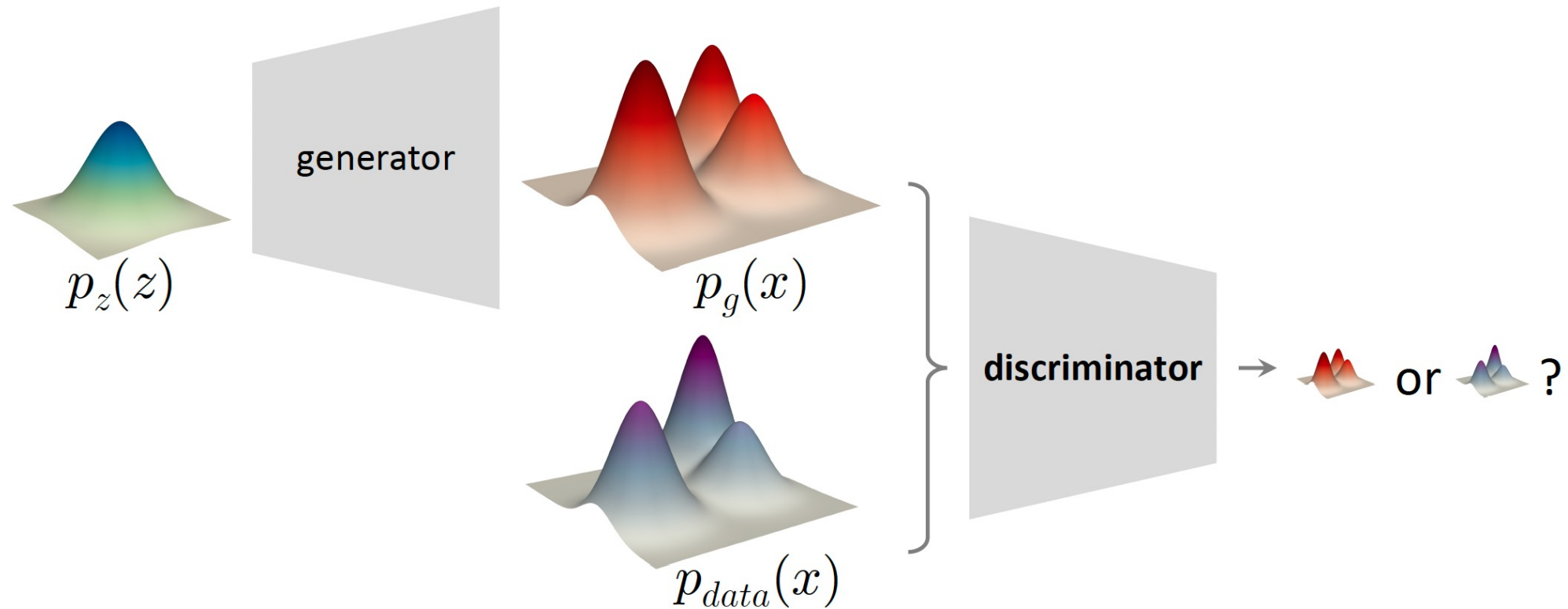
Generative Adversarial Networks:
Measure distribution difference in another way

Representing **distribution difference** by a neural network



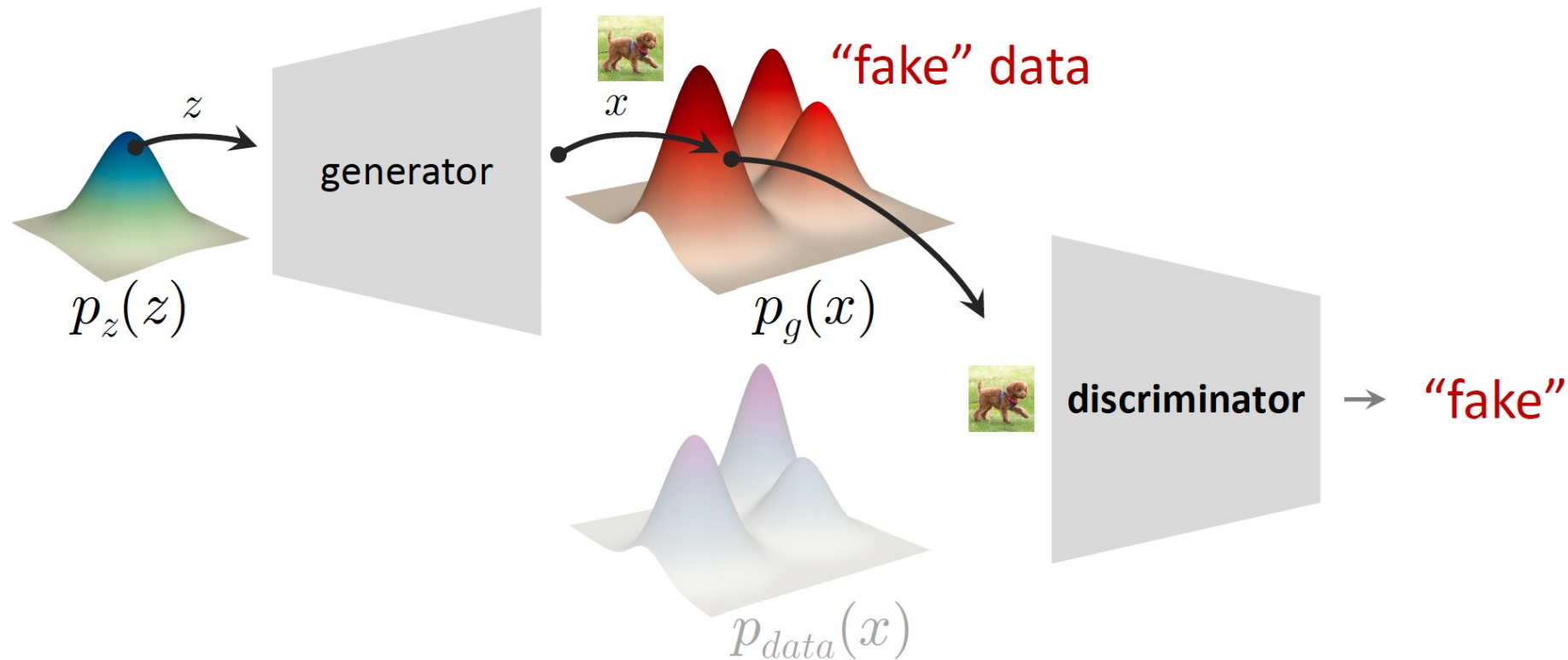
Representing distribution difference by a neural network **without explicitly measuring the difference**

- Train a classifier to distinguish distributions



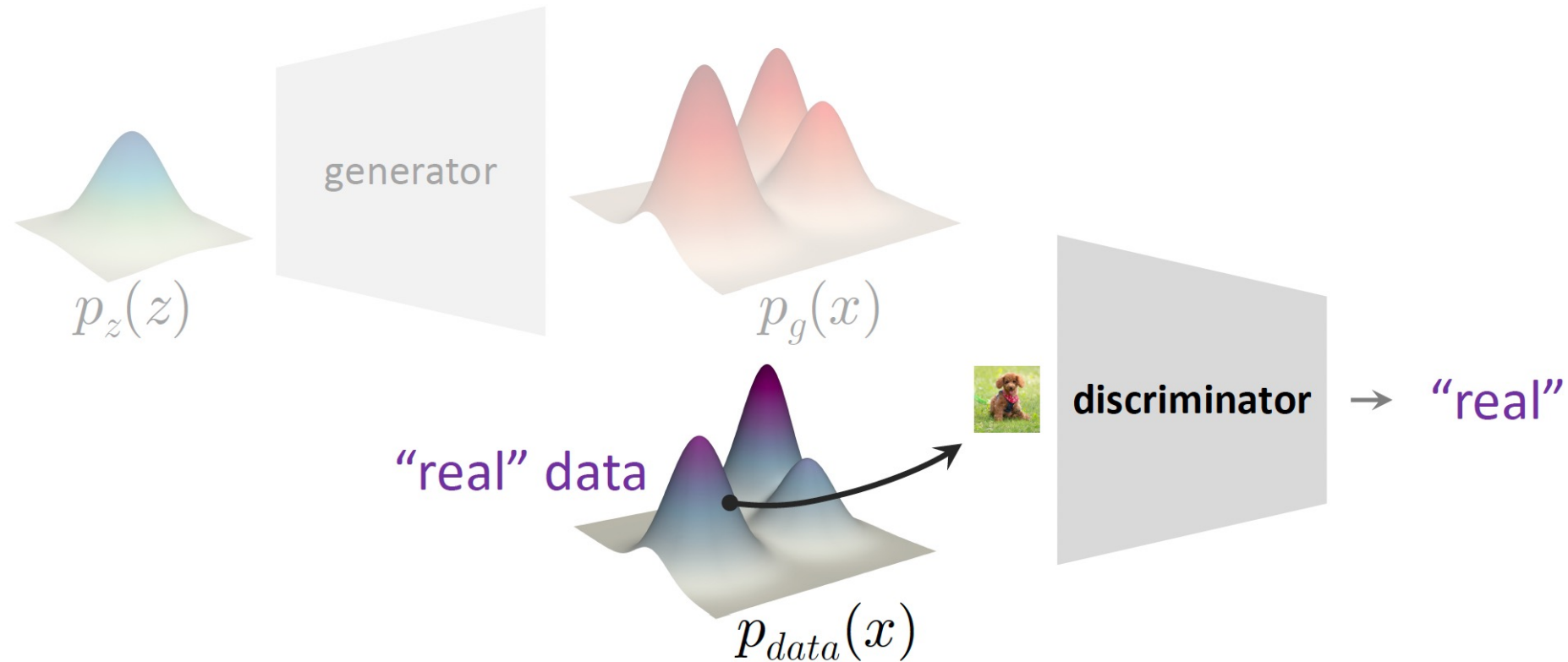
Representing distribution difference by a neural network without explicitly measuring the difference

- Train a classifier to distinguish distributions **at sample-level**

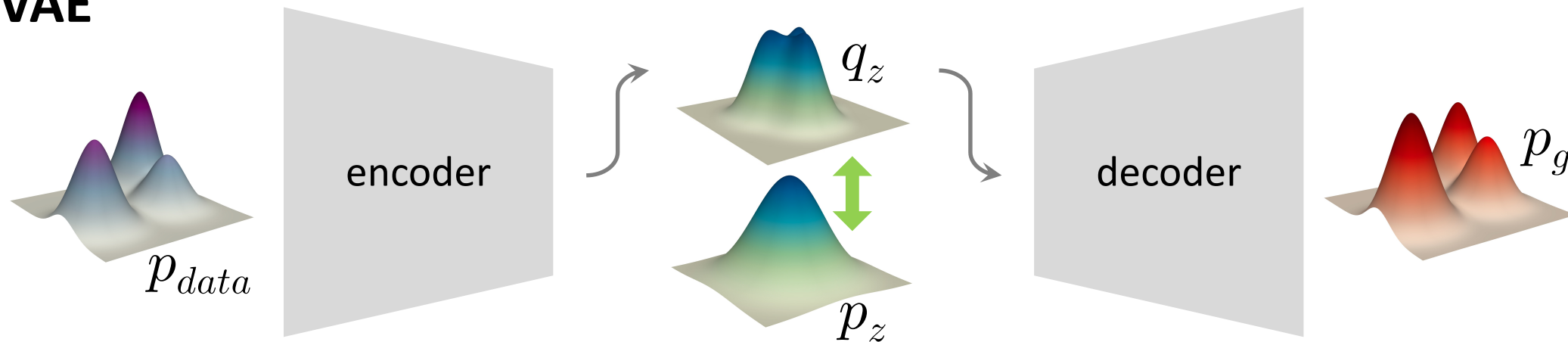


Representing distribution difference by a neural network without explicitly measuring the difference

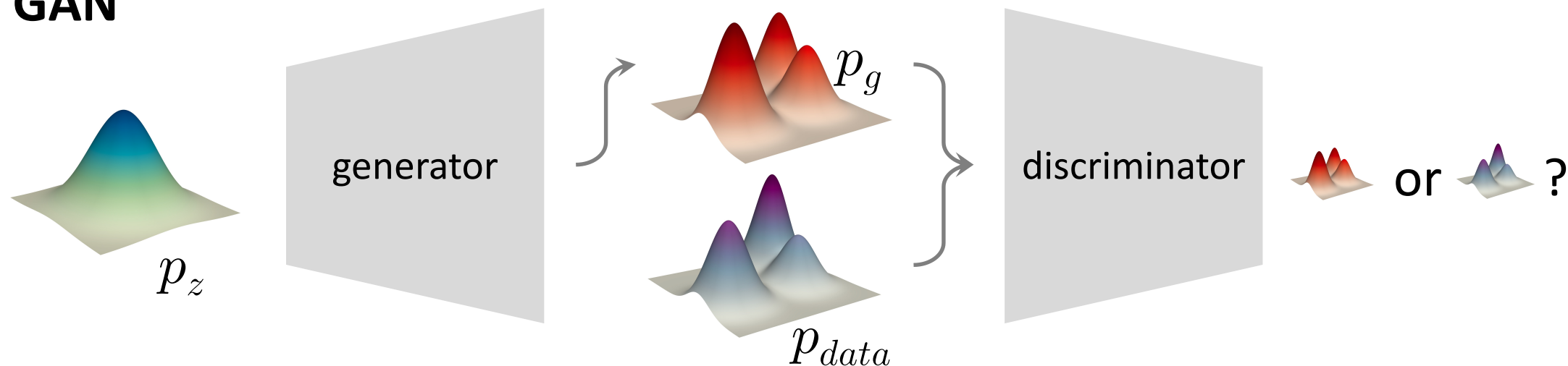
- Train a classifier to distinguish distributions **at sample-level**



VAE



GAN

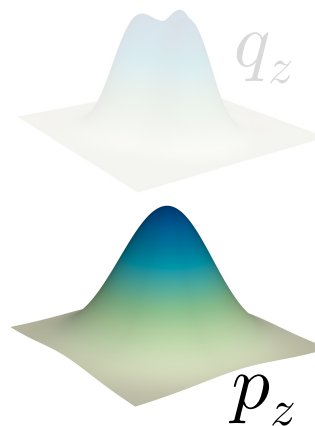


VAE

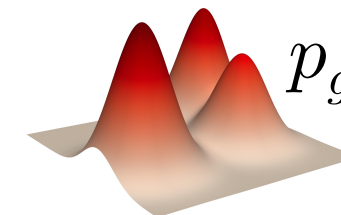
generation



encoder

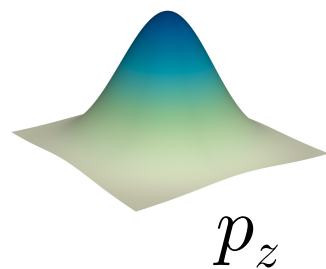


decoder

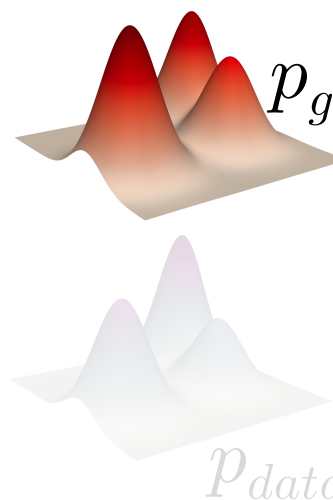


GAN

generation



generator



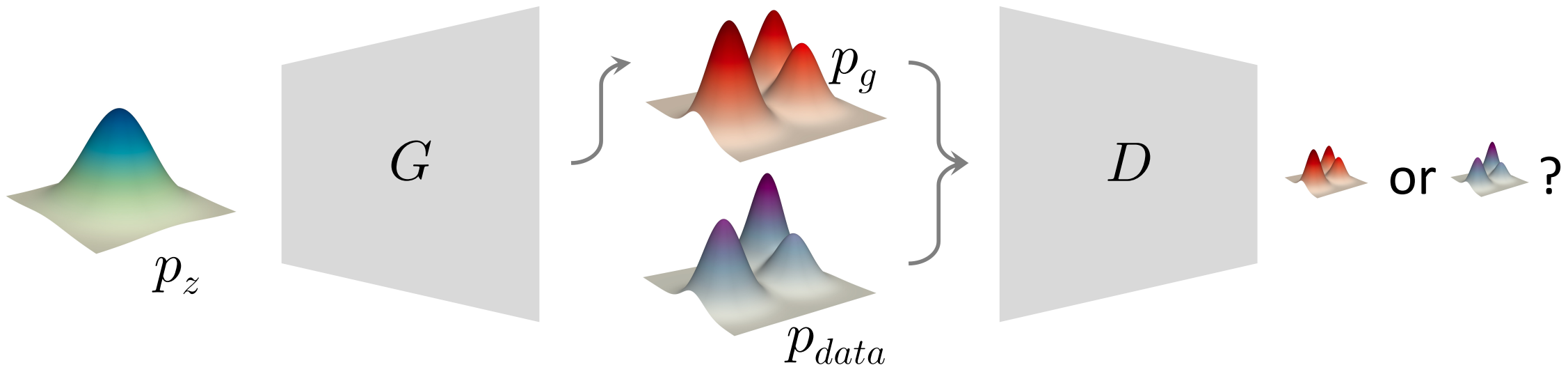
discriminator



Objective of GAN

$$\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

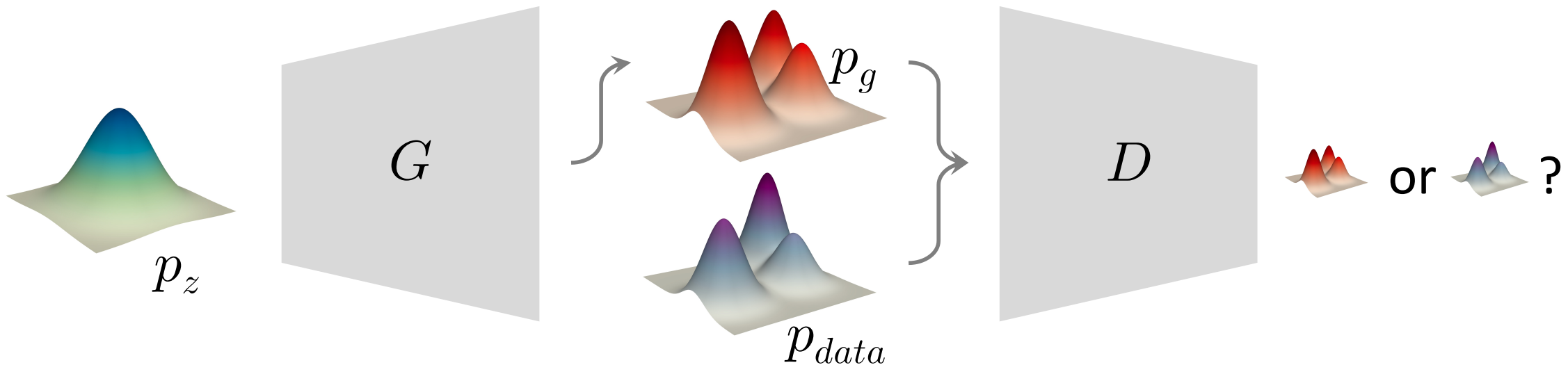
min-max process
(vs. EM's max-max process)



Objective of GAN

$$\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log \underline{D}(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - \underline{\underline{D}}(G(z)))]$$

D -step: fix G , optimize D

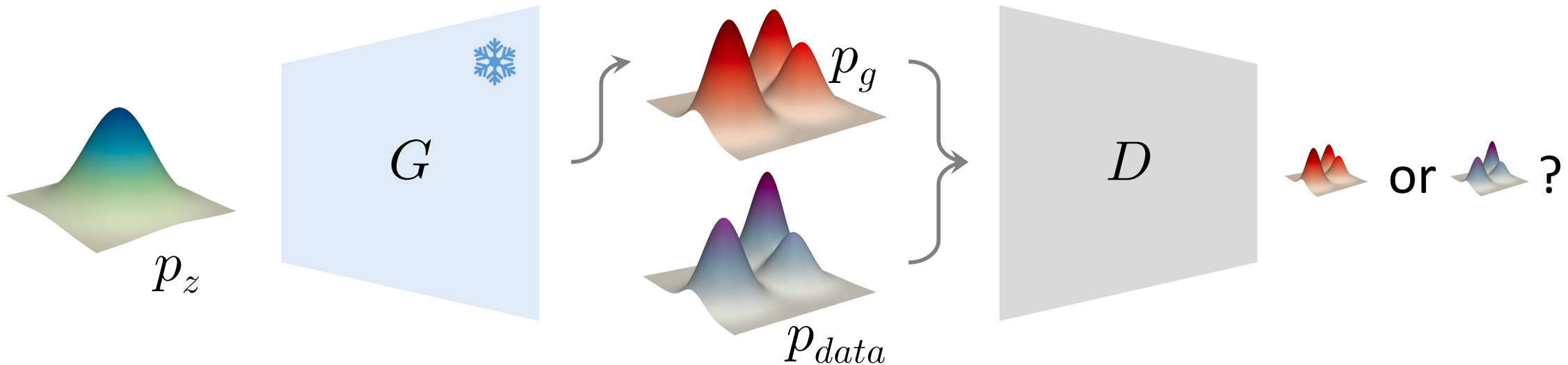


Objective of GAN

$$\max_D \mathcal{L}(D) = \mathbb{E}_{x \sim p_{\text{data}}} [\log \underbrace{D(x)}_{\text{push to 1}}] + \mathbb{E}_{z \sim p_z} [\log(1 - \underbrace{D(G(z))}_{\text{push to 0}})]$$

D -step: fix G , optimize D

- D to classify real or fake
- binary logistic regression (sigmoid + cross-entropy)

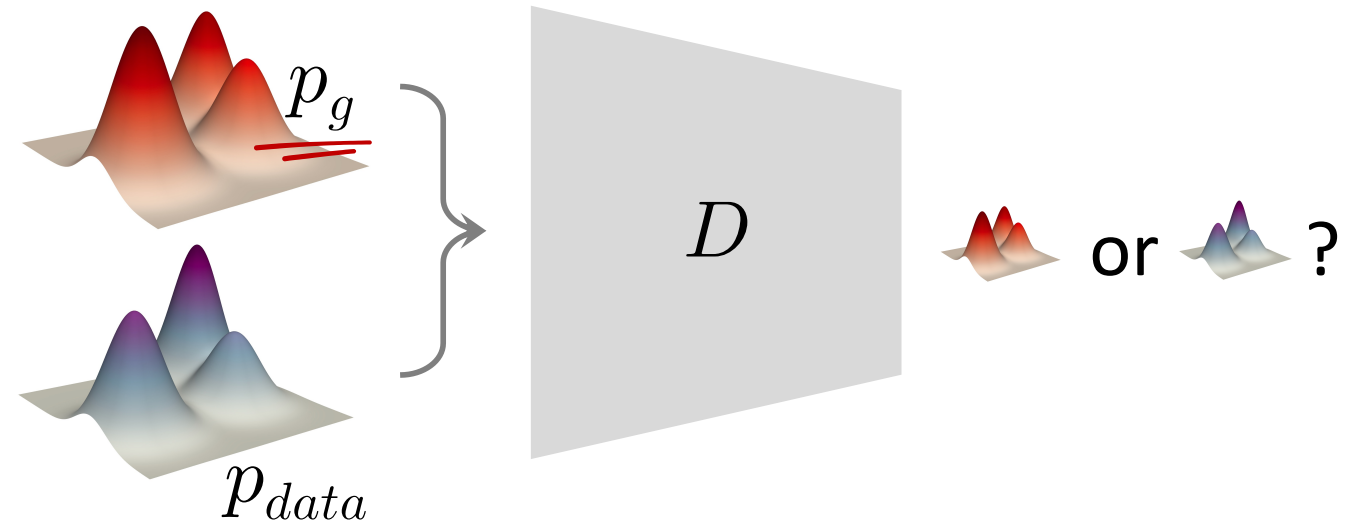


Objective of GAN

$$\max_D \mathcal{L}(D) = \mathbb{E}_{x \sim p_{\text{data}}} [\log \underbrace{D(x)}_{\text{push to 1}}] + \mathbb{E}_{z \sim p_z} [\log(1 - \underbrace{D(G(z))}_{\text{push to 0}})]$$

D -step: fix G , optimize D

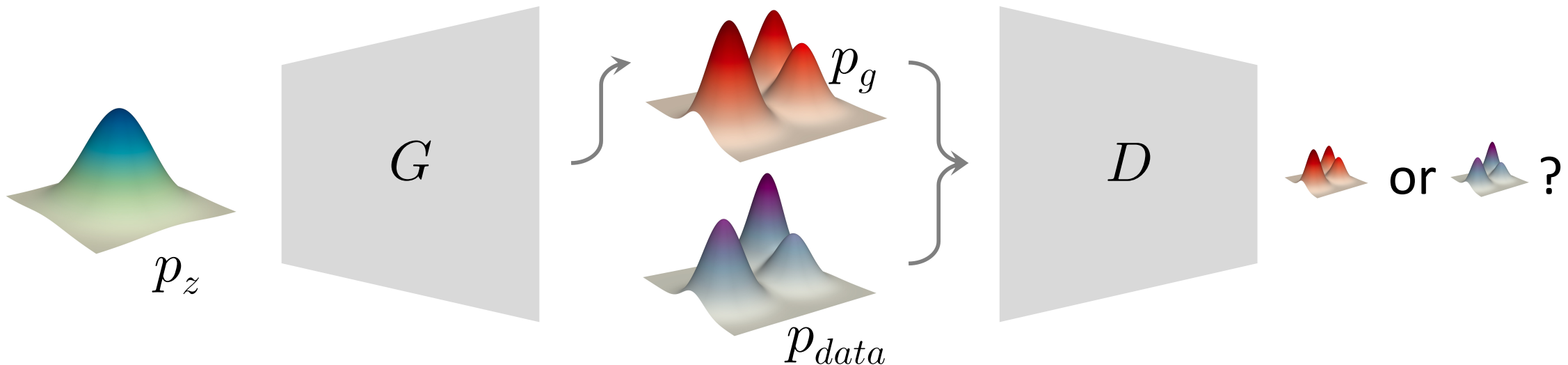
- D to classify real or fake
- binary logistic regression (sigmoid + cross-entropy)



Objective of GAN

$$\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

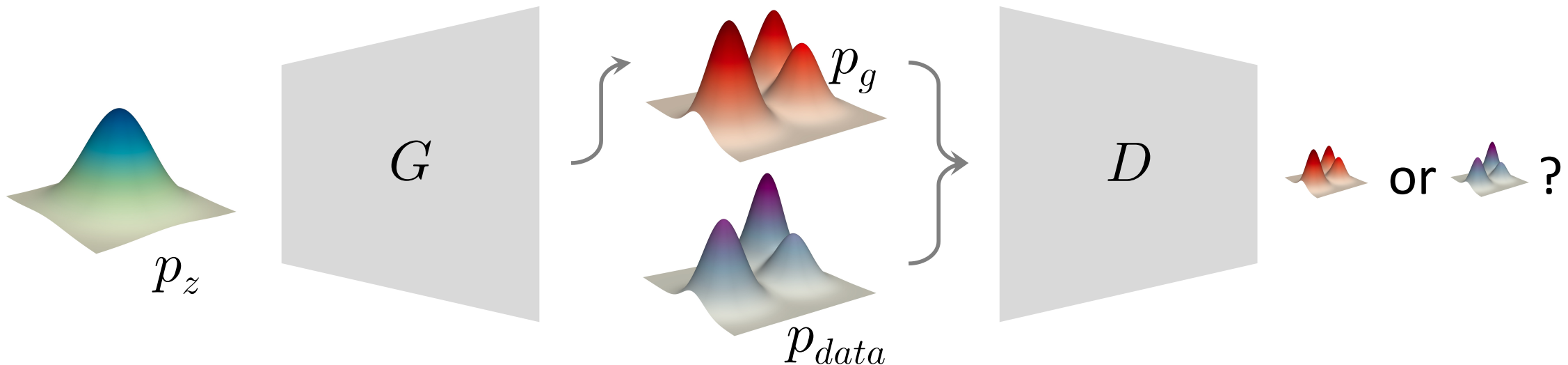
G -step: fix D , optimize G



Objective of GAN

$$\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

G -step: fix D , optimize G

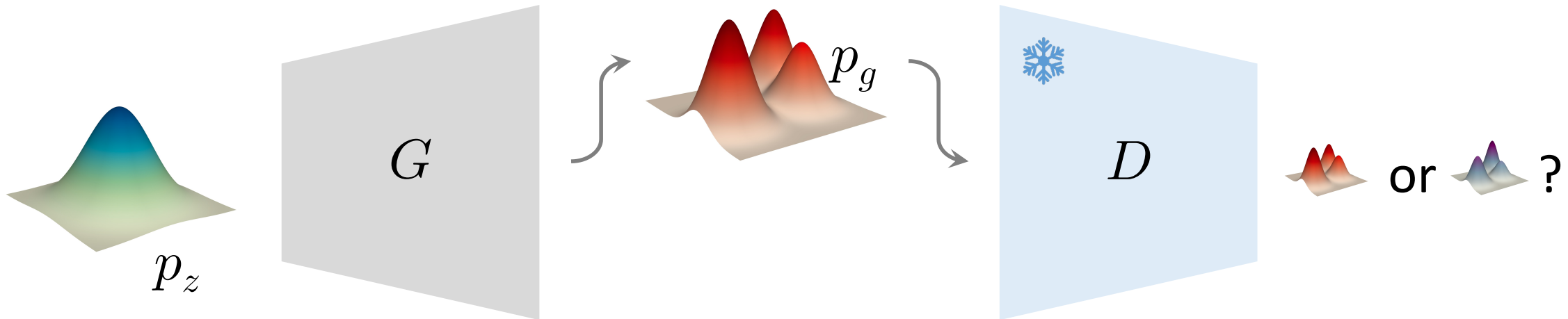


Objective of GAN

$$\min_G \mathcal{L}(G) = \mathbb{E}_{z \sim p_z} [\log(1 - \underbrace{D(G(z))}_{\text{push to 1}})]$$

G -step: fix D , optimize G

- generate fake data such that D classifies it as “real”
- G to “confuse” D

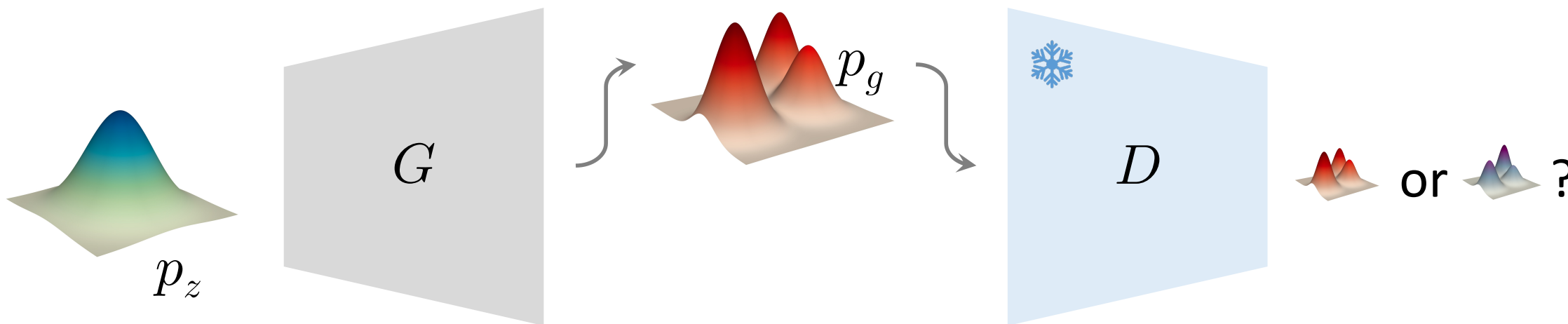


Objective of GAN

a “flip” trick: $\max_G \mathcal{L}(G) = \mathbb{E}_{z \sim p_z} [\log(\underbrace{1 - D(G(z))}_{\text{push to 1}})]$

G -step: fix D , optimize G

- generate fake data such that D classifies it as “real”
- G to “confuse” D



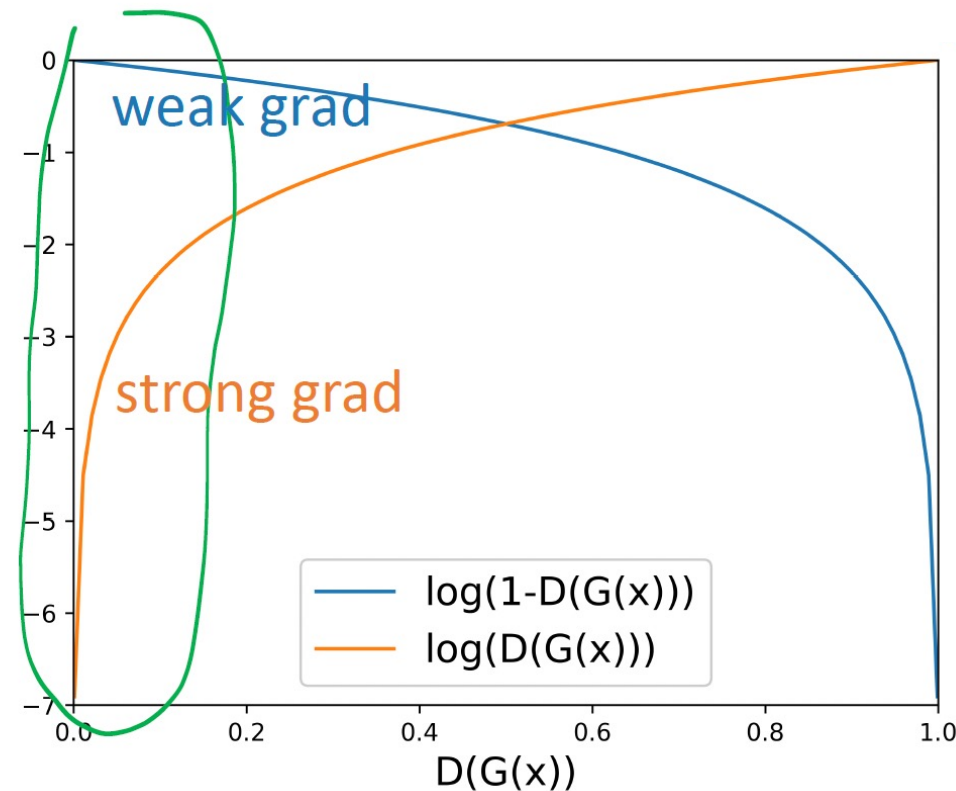
Objective of GAN

a “flip” trick:

$$\max_D \min_G \mathcal{L}(G) = \mathbb{E}_{z \sim p_z} [\log(\underbrace{1 - D(G(z))}_{\text{push to 1}})]$$

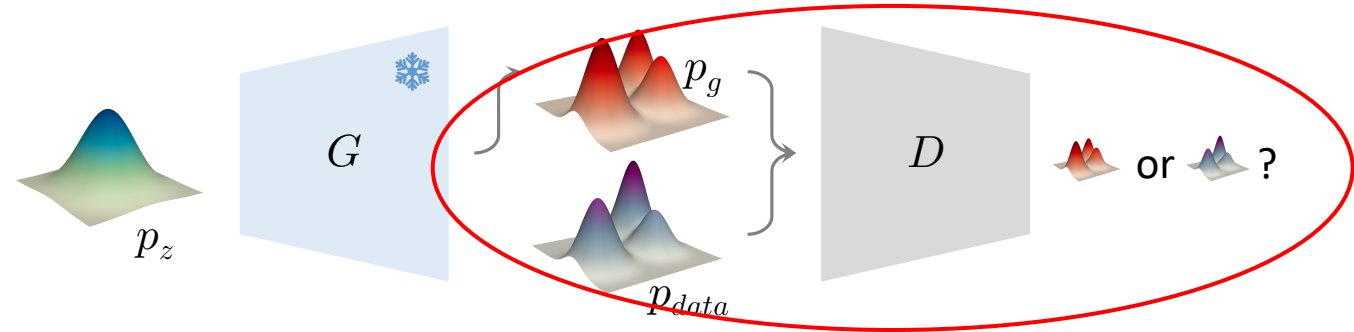
Early in training:

- G is poor
- $D(G)$ is near 0



Objective of GAN: Pseudo-code

- **D-step**



Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**
 for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

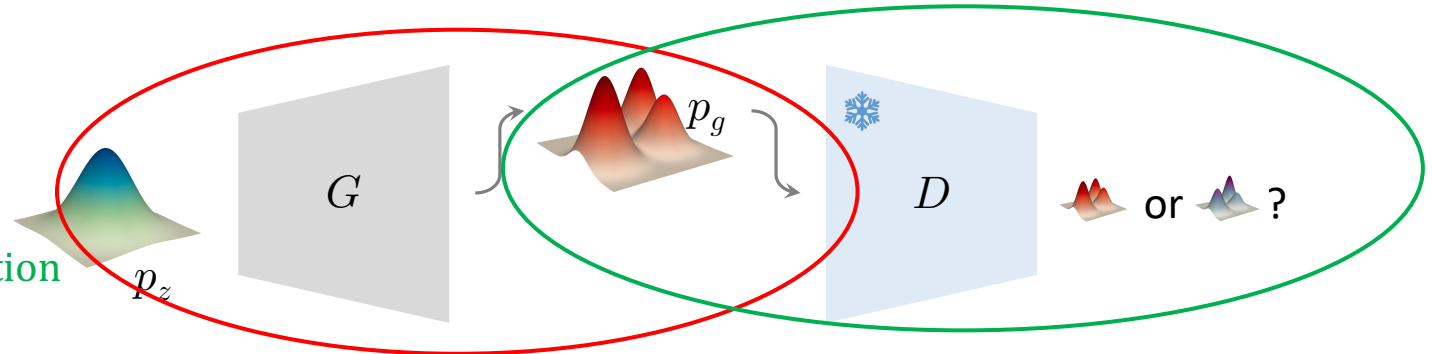
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Objective of GAN: Pseudo-code

- **G-step**
- **D serves as a parameterized loss function**



Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Objective of GAN: Pseudo-code

- Update frequencies of D and G can be different (a hyper-parameter)

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

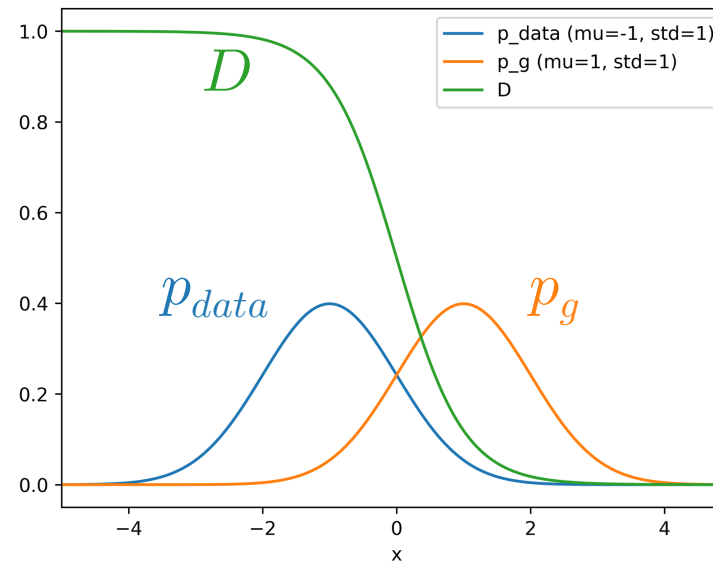
end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Objective of GAN: Theoretical Results

1. For any given G , the optimal D is:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

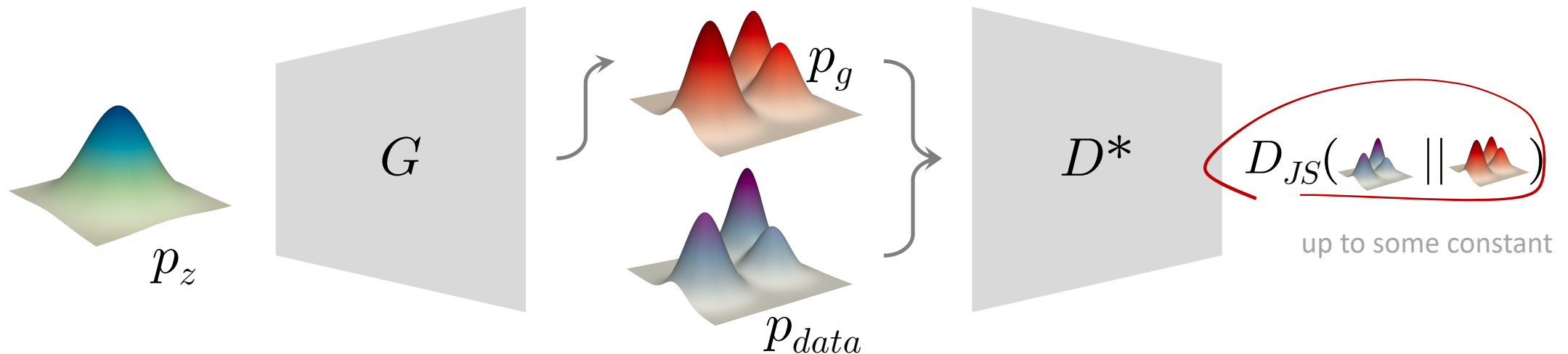


Objective of GAN: Theoretical Results

2. With the optimal D_G , the objective function is:

$$\mathcal{L}(D^*, G) = 2D_{JS}(p_{\text{data}} || p_g) - 2 \log 2$$

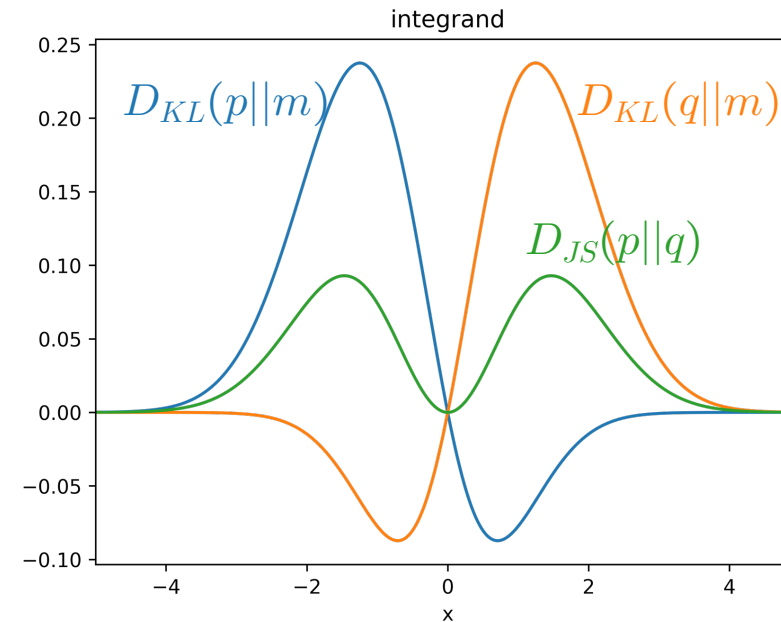
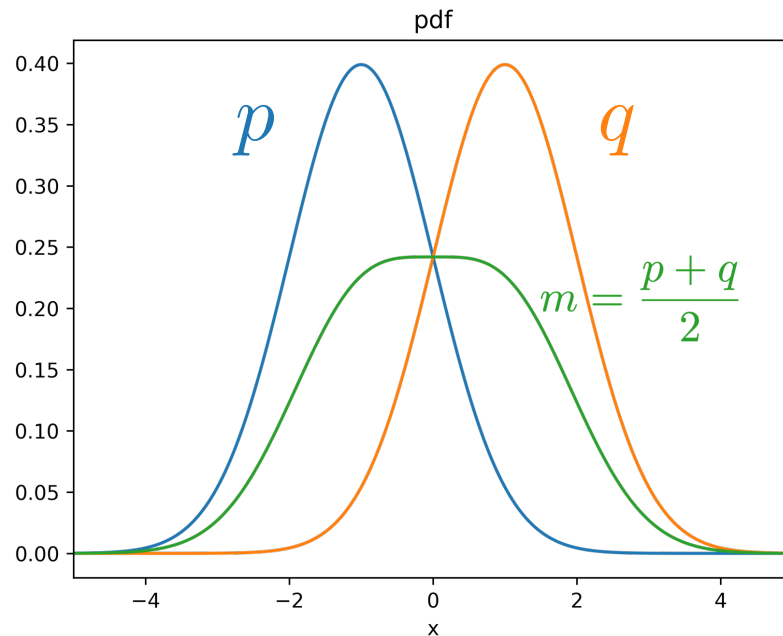
where D_{JS} is Jensen–Shannon divergence



Objective of GAN: Theoretical Results

D_{JS} : “total divergence to the average”

$$D_{JS}(p||q) \triangleq \frac{1}{2}D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2}D_{KL}(q||\frac{p+q}{2})$$



Objective of GAN: Theoretical Results

D_{JS} : “total divergence to the average”

$$D_{JS}(p||q) \triangleq \frac{1}{2}D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2}D_{KL}(q||\frac{p+q}{2})$$

Properties:

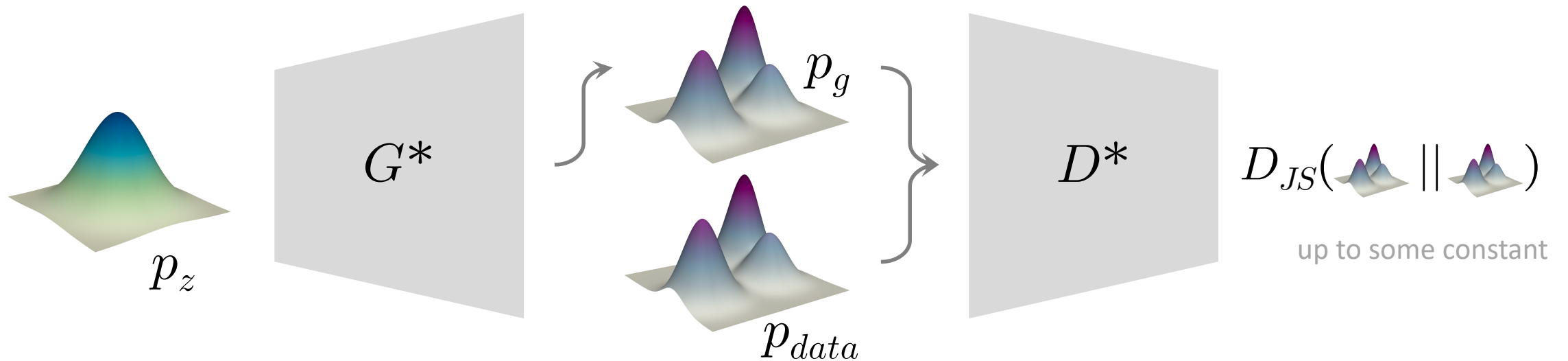
- D_{JS} is symmetric; D_{KL} is not
- D_{JS} is bounded: $[0, \log 2]$; D_{KL} is unbounded: $[0, \infty)$
- D_{JS} is more stable

Objective of GAN: Theoretical Results

3. Global optimality is achieved at $p_g = p_{data}$

$$\mathcal{L}(D^*, G^*) = 2D_{JS}(p_{data} || p_g) - 2\log 2$$

~~$= 2D_{JS}(p_{data} || p_g) - 2\log 2$~~
 $\Rightarrow$



Objective of GAN: Theoretical Results

1. For any given G , the optimal D is:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

2. With optimal D_G , GAN optimizes for Jensen–Shannon divergence:

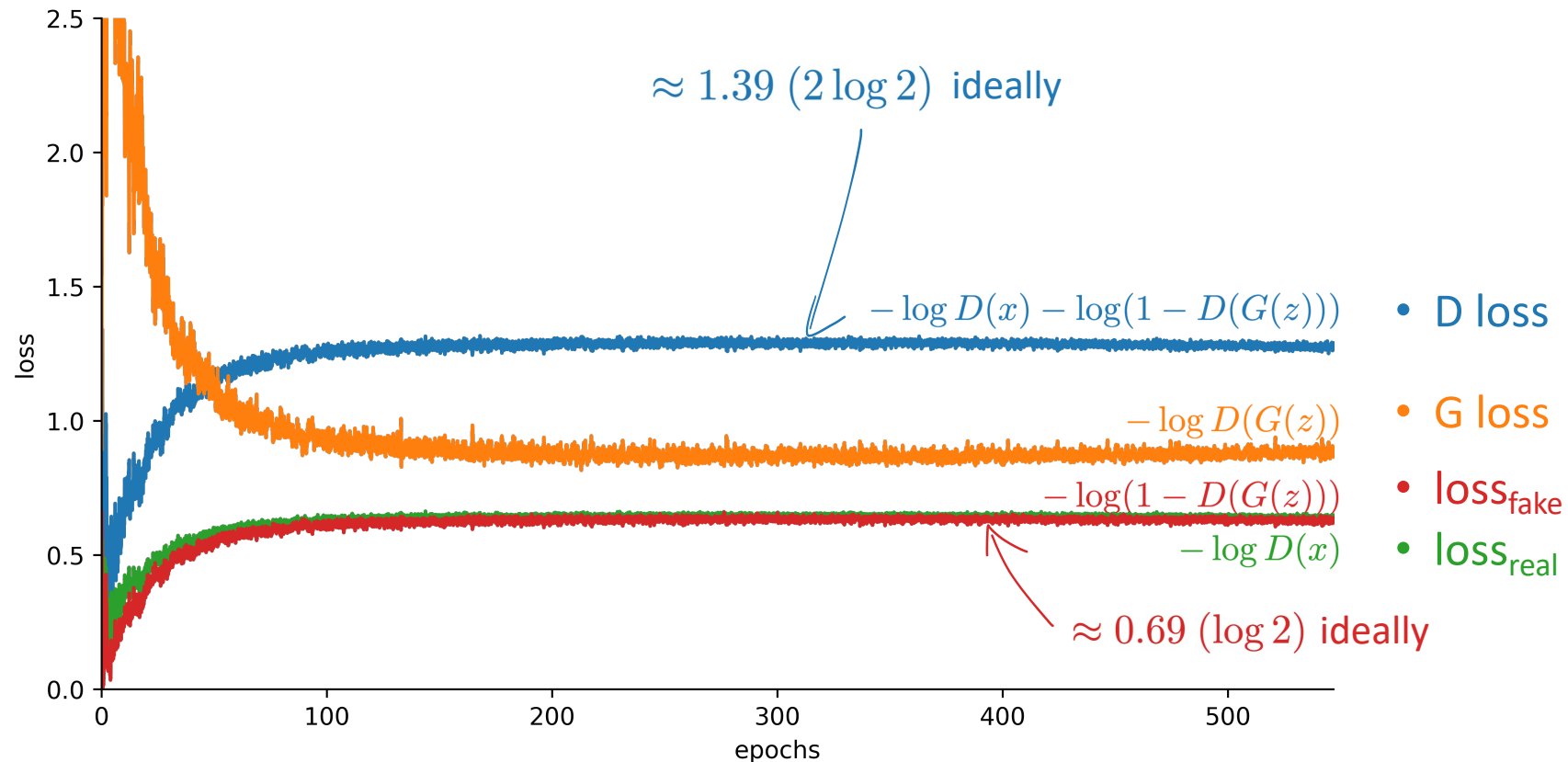
$$\mathcal{L}(D^*, G) = 2D_{JS}(p_{\text{data}} \| p_g) - 2 \log 2$$

3. Global optimality is achieved at $p_g = p_{\text{data}}$

$$\mathcal{L}(D^*, G^*) = -2 \log 2$$

GAN Training

Running example: MNIST

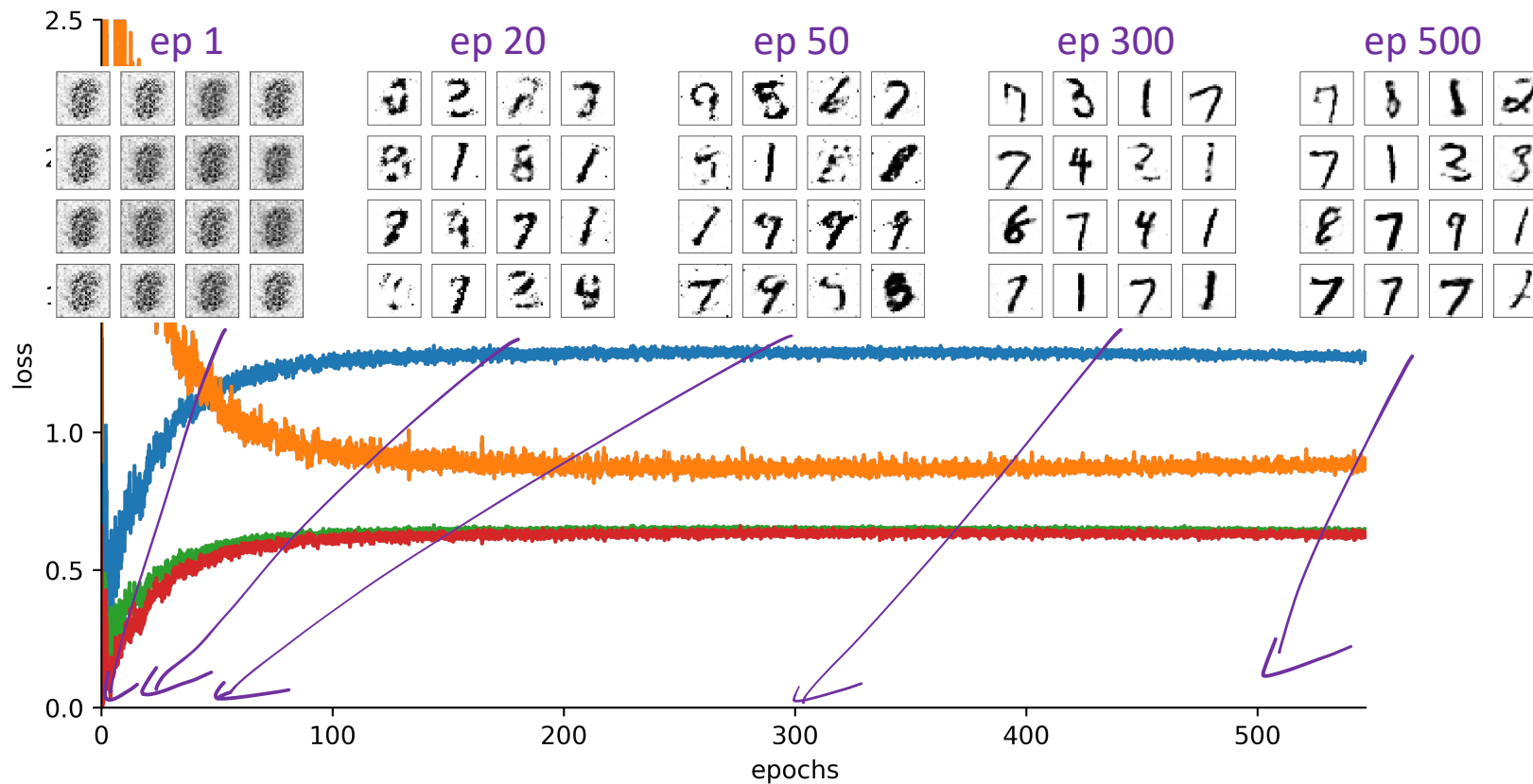


*All objectives are negative of their original form

Code adapted from: <https://github.com/prcastro/pytorch-gan/tree/master>

GAN Training

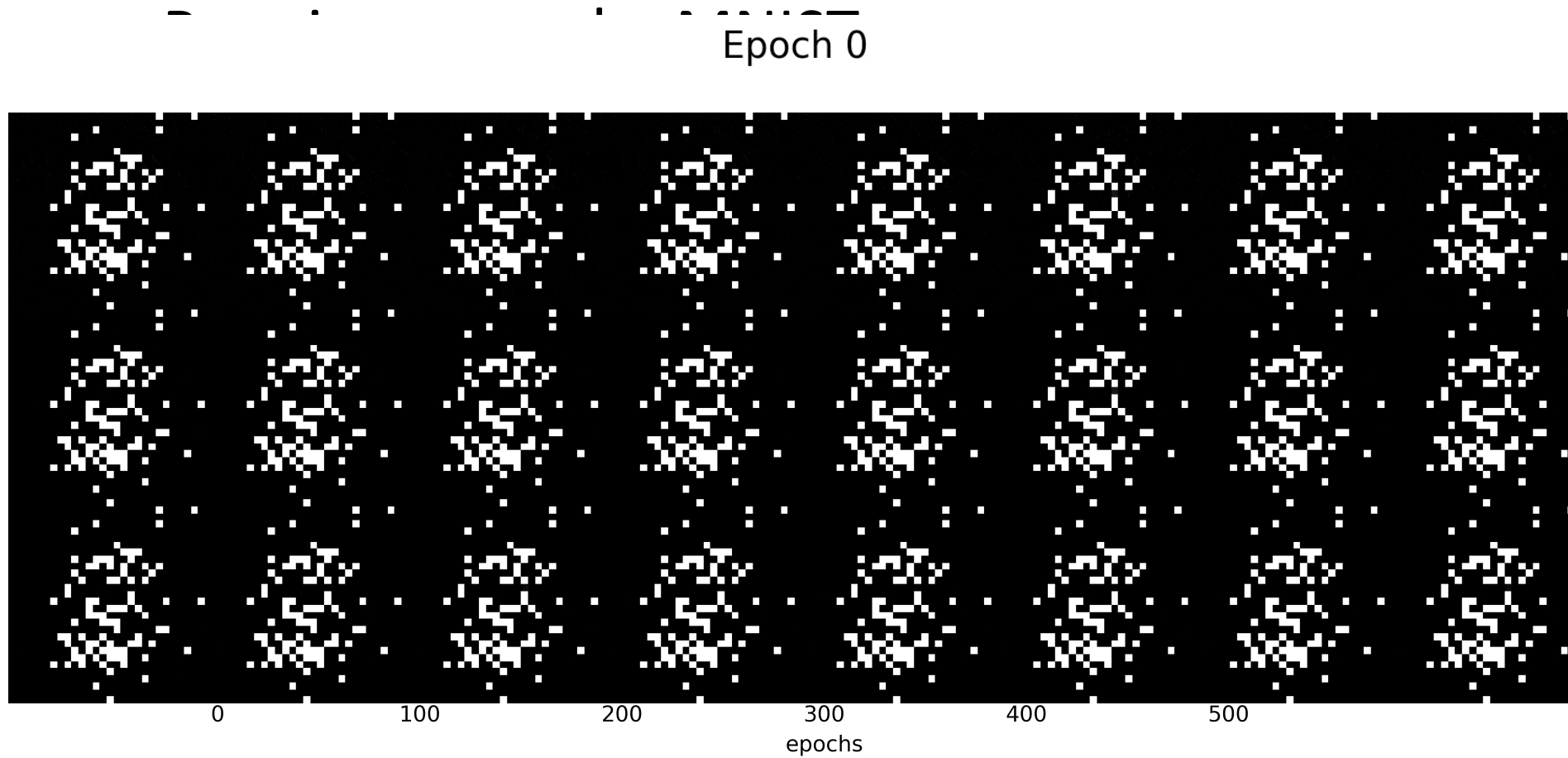
Running example: MNIST



*All objectives are negative of their original form

Code adapted from: <https://github.com/prcastro/pytorch-gan/tree/master>

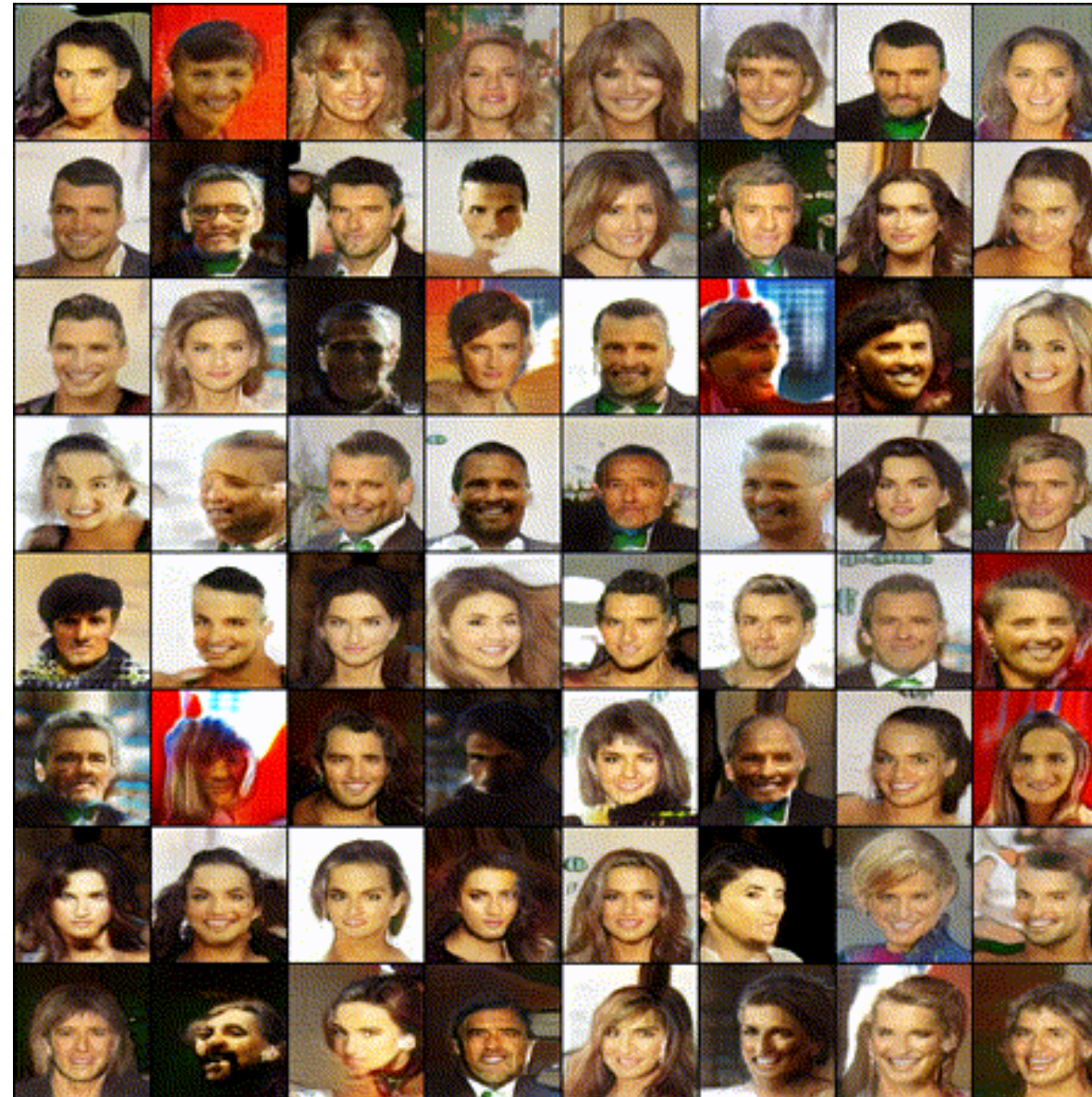
GAN Training



*All objectives are negative of their original form

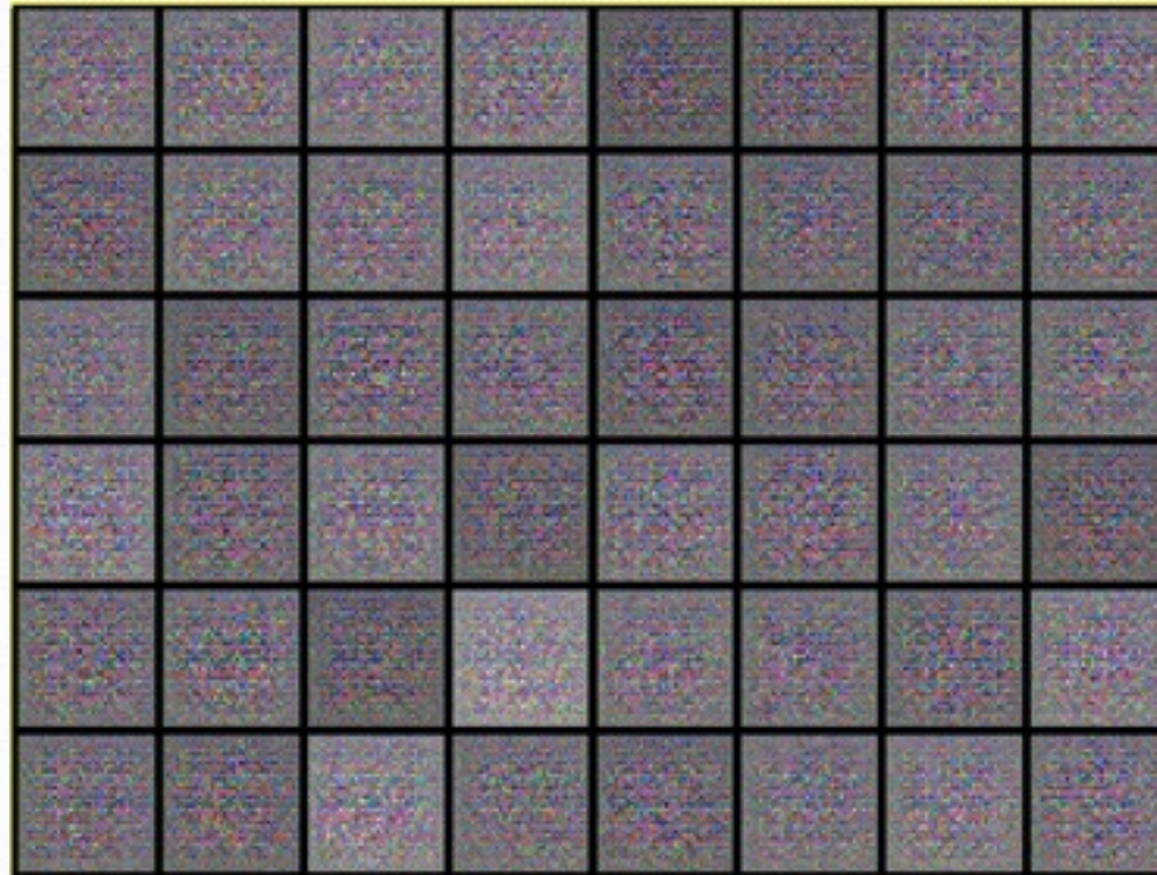
Code adapted from: <https://github.com/prcastro/pytorch-gan/tree/master>

GAN Training



GAN Training

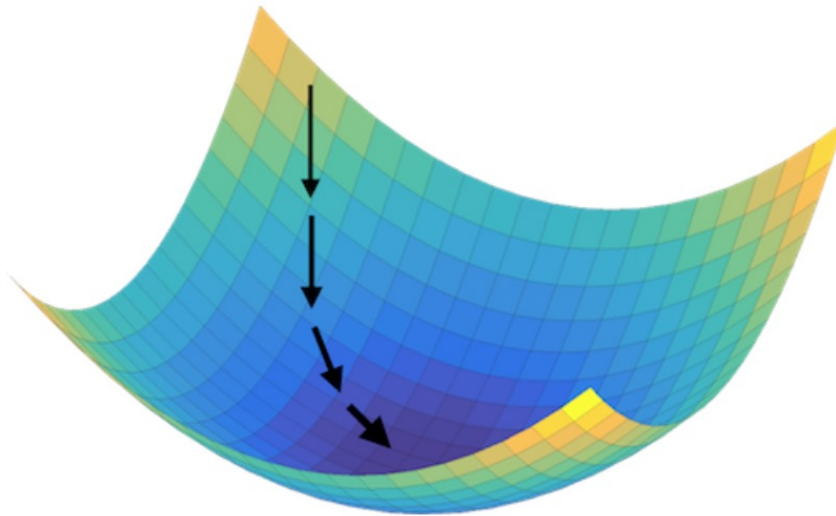
- Things can go wrong ...



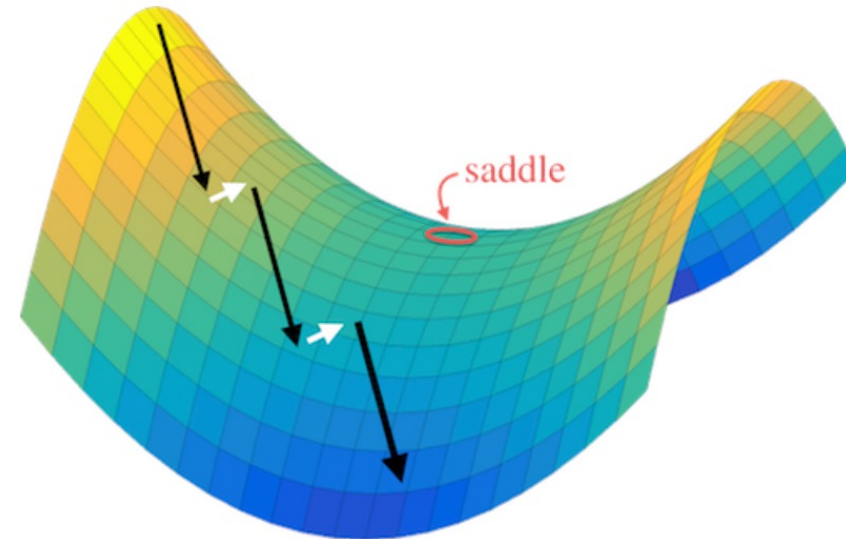
GAN Training: Challenges

- Hard to achieve Nash equilibrium (unstable training dynamics)

Standard Neural Net

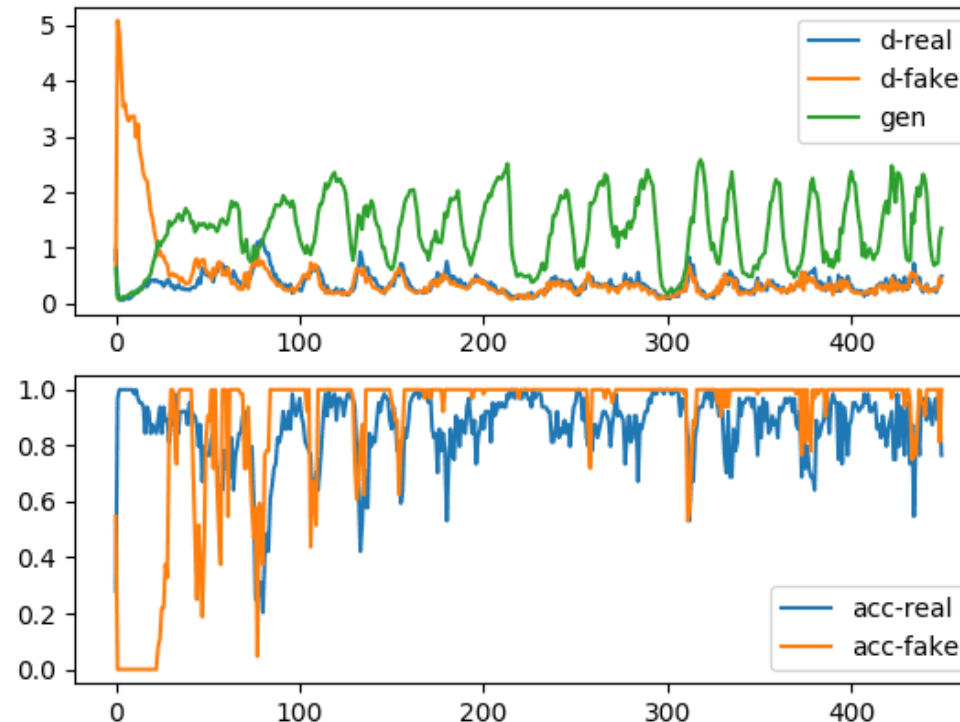


Adversarial Net



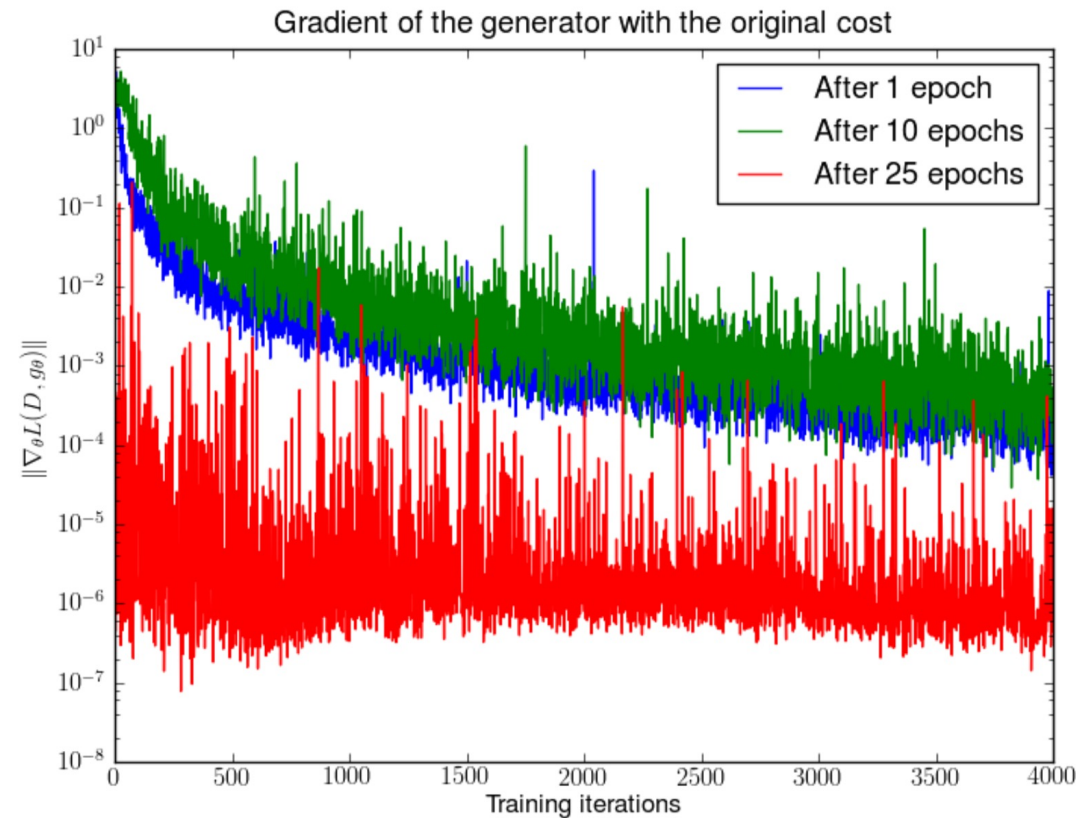
GAN Training: Challenges

- Hard to achieve Nash equilibrium (unstable training dynamics)
 - Discriminator is **too strong** for Generator
 - Generator is **too strong** for Discriminator
 - Large **variance** in loss and gradients
 - **Sensitivity** to hyperparameters and network architectures
 - ...



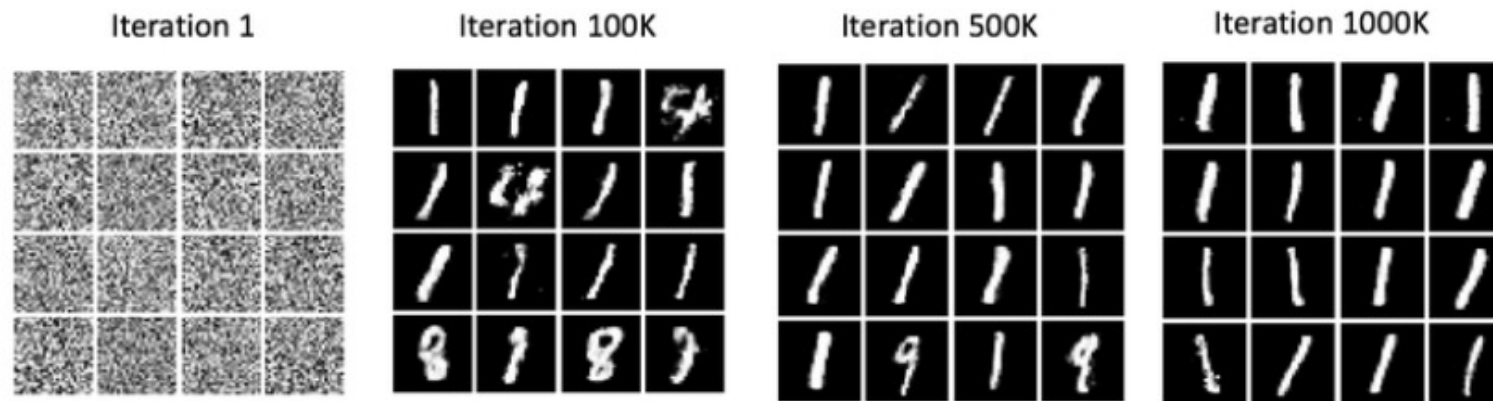
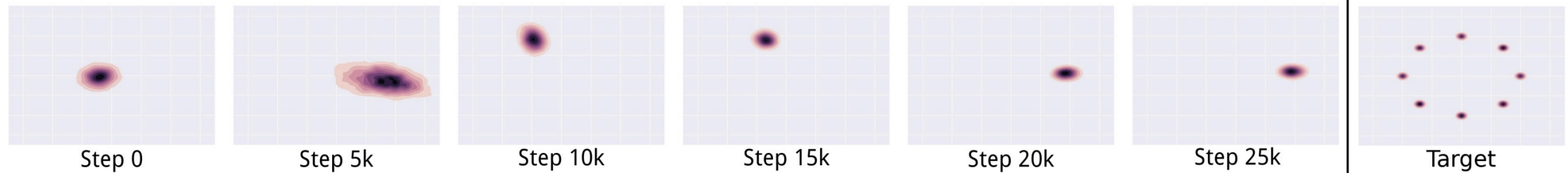
GAN Training: Challenges

- Hard to achieve Nash equilibrium (unstable training dynamics)
- **Vanishing gradient**
 - the gradient norms decay quickly (in log scale)



GAN Training: Challenges

- Hard to achieve Nash equilibrium (unstable training dynamics)
- Vanishing gradient
- **Mode collapse**



GAN Training is hard

- Improved training techniques
 - Feature Matching
 1. Train the generator to match the expected value of the features on an intermediate layer of the discriminator.
 2. Empirical results indicate that feature matching is indeed effective in situations where regular GAN becomes unstable.

GAN Training is hard

- Improved training techniques
 - Feature Matching
 - Minibatch Discrimination

1. allow the discriminator to look at multiple data examples in combination and perform minibatch discrimination.
2. any discriminator model that looks at multiple examples in combination, rather than in isolation, could potentially help avoid collapse of the generator.

GAN Training is hard

- **Improved training techniques**
 - Feature Matching
 - Minibatch Discrimination
 - Historical Average

1. Modify each player's cost function to include the following term:

$$||\boldsymbol{\theta} - \frac{1}{t} \sum_{i=1}^t \boldsymbol{\theta}[i]||^2$$

2. The historical average of the parameters can be updated in an online fashion, so this learning rule scales well to long time series.

GAN Training is hard

- Improved training techniques
 - Feature Matching
 - Minibatch Discrimination
 - Historical Average
 - Better measurement for distance difference

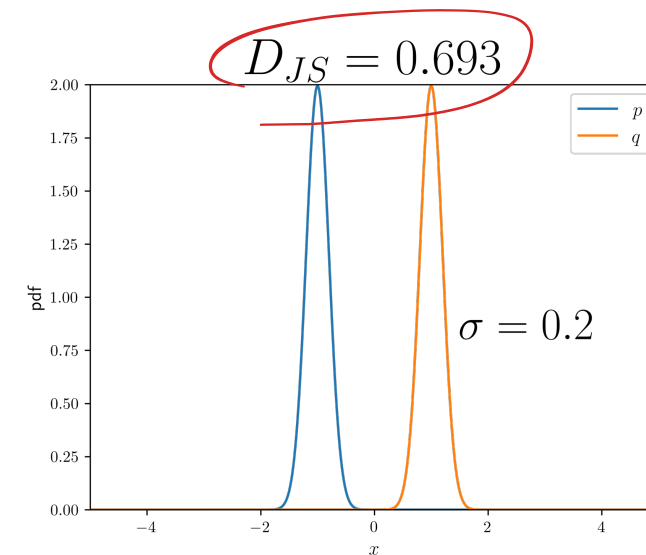
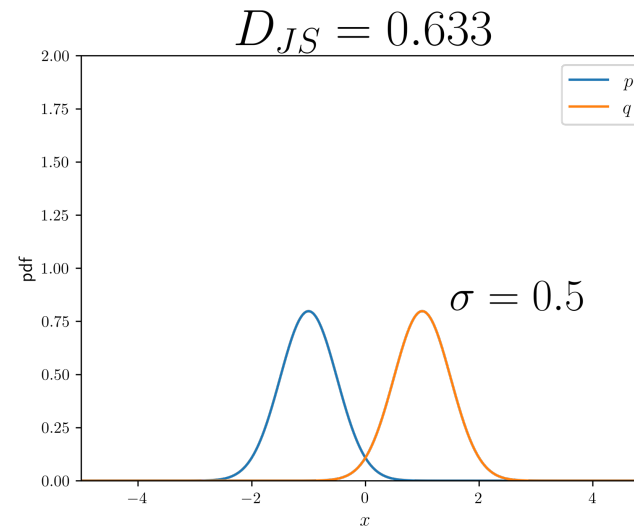
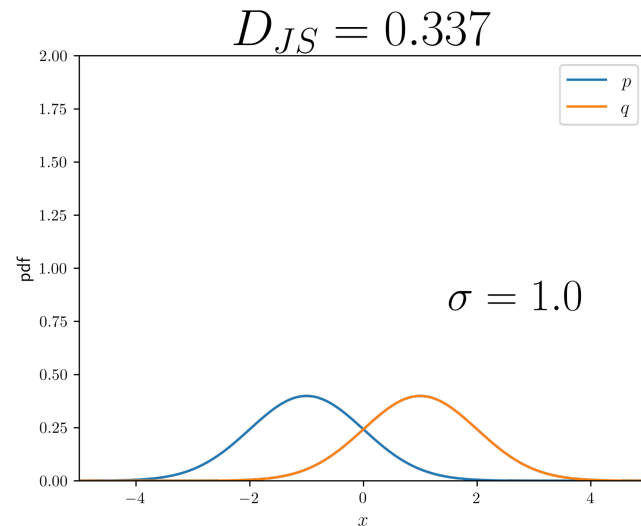
Wasserstein GAN

- Problem of D_{JS} : if p and q don't overlap, D_{JS} is a constant, i.e., no gradient

With the optimal D_G , the objective function is:

$$\mathcal{L}(D^*, G) = 2D_{JS}(p_{\text{data}} || p_g) - 2 \log 2$$

where D_{JS} is Jensen–Shannon divergence



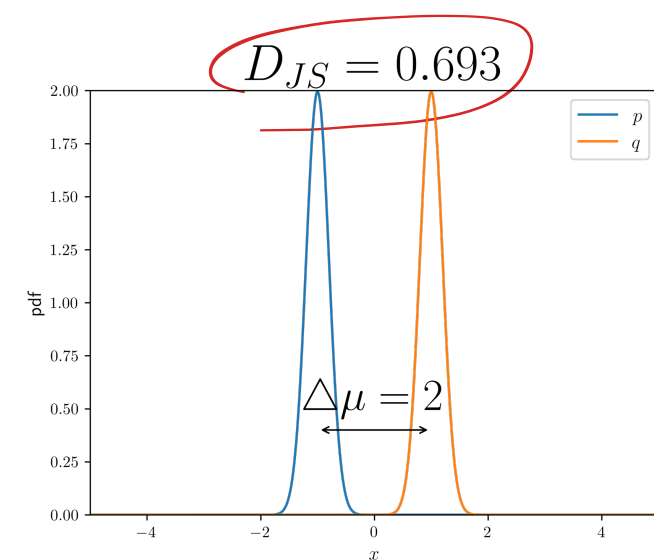
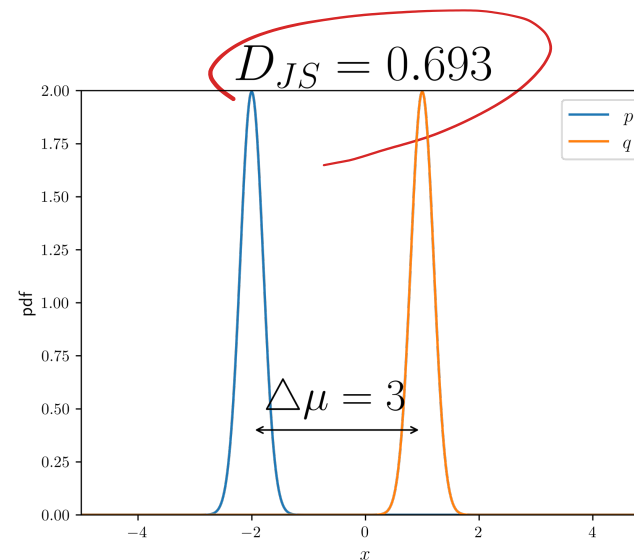
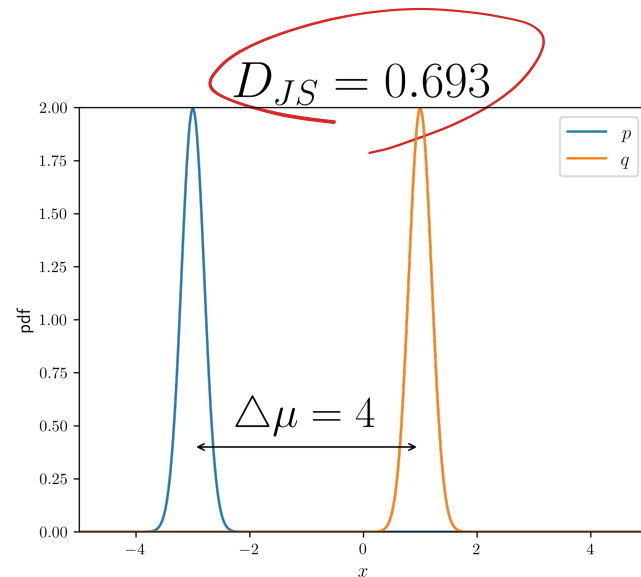
Wasserstein GAN

- Problem of D_{JS} : if p and q don't overlap, D_{JS} is a constant, i.e., no gradient

With the optimal D_G , the objective function is:

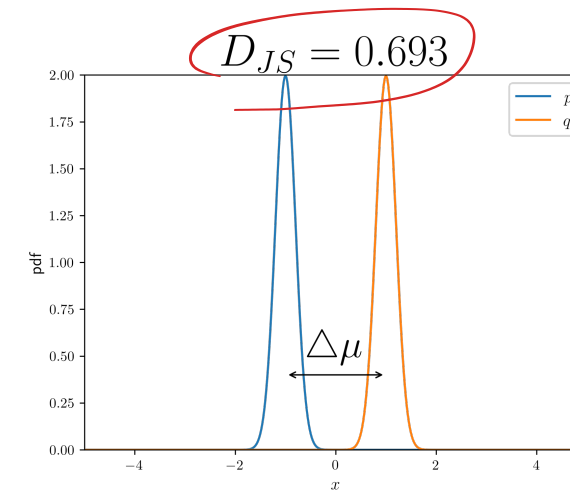
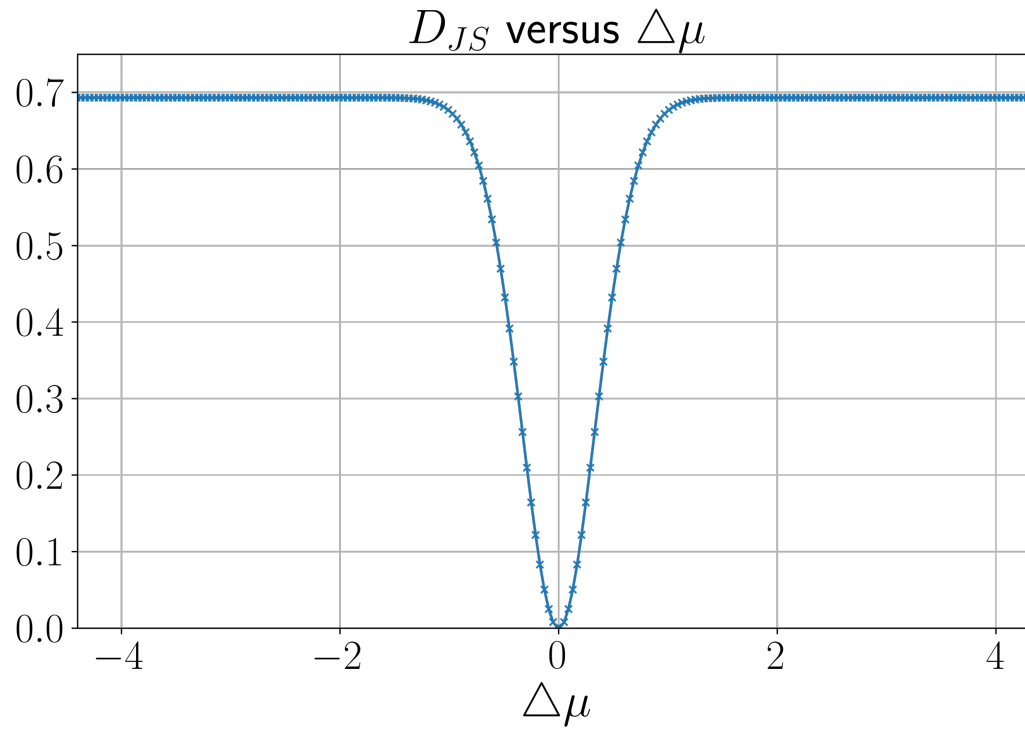
$$\mathcal{L}(D^*, G) = 2D_{JS}(p_{\text{data}} || p_g) - 2 \log 2$$

where D_{JS} is Jensen–Shannon divergence



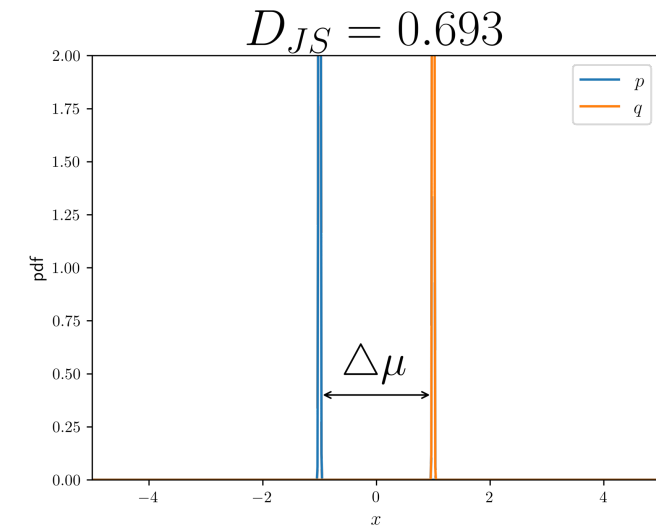
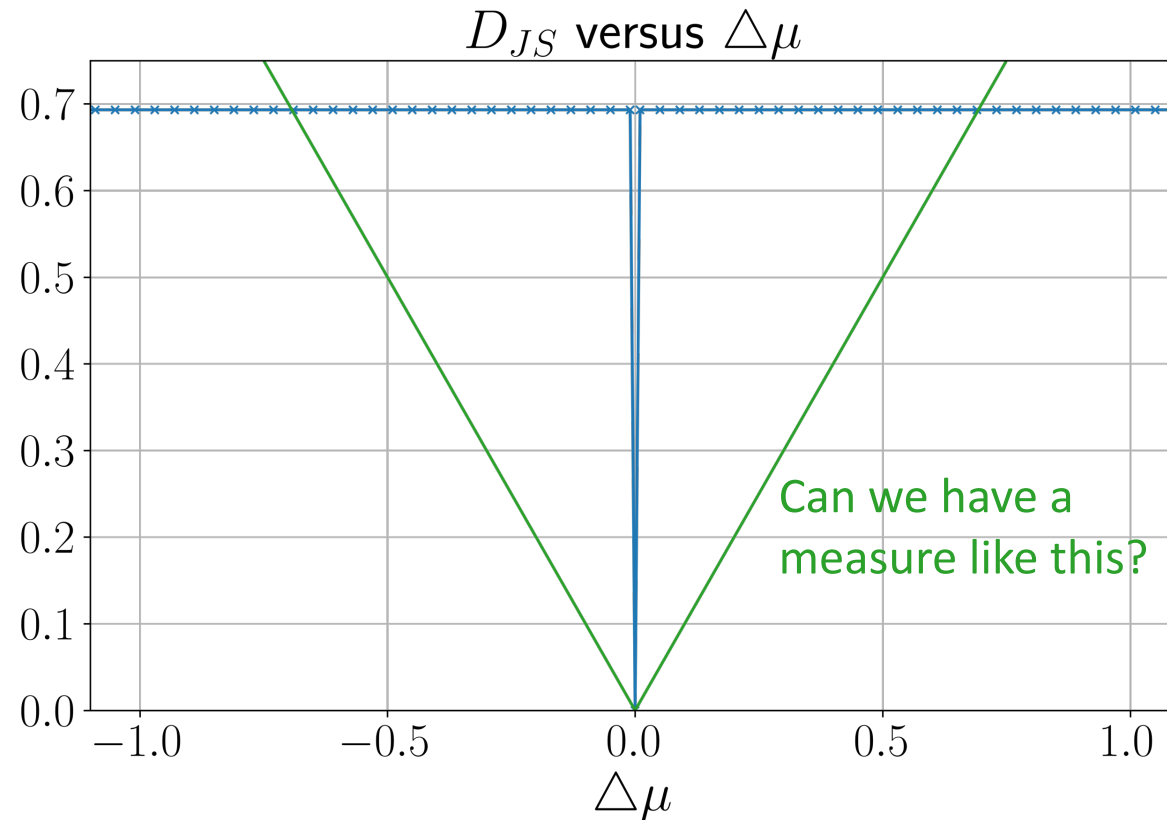
Wasserstein GAN

- Problem of D_{JS} : if p and q don't overlap, D_{JS} is a constant, i.e., no gradient
- D_{JS} is useful only if p and q are close



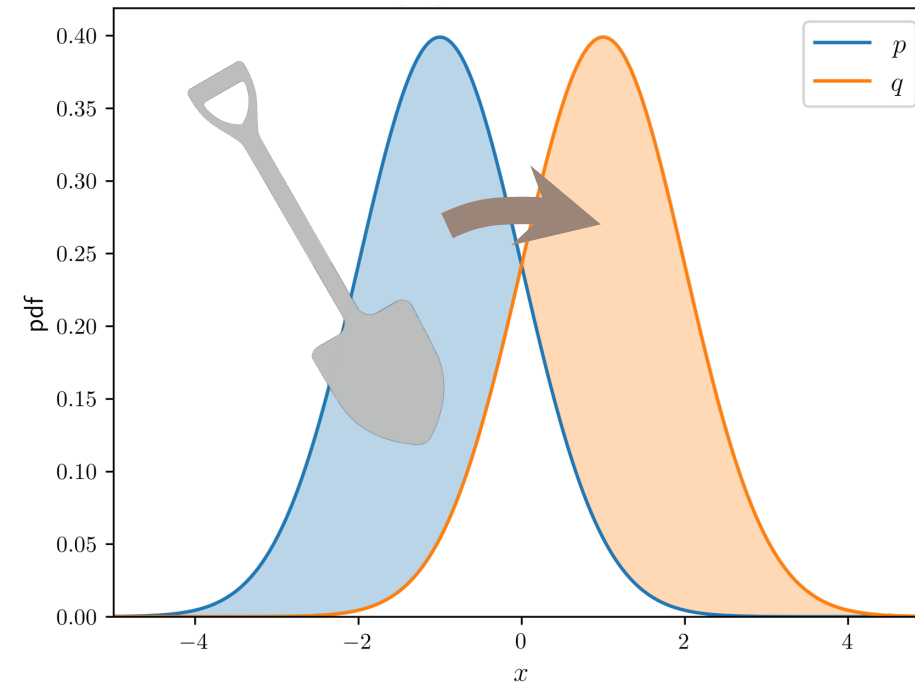
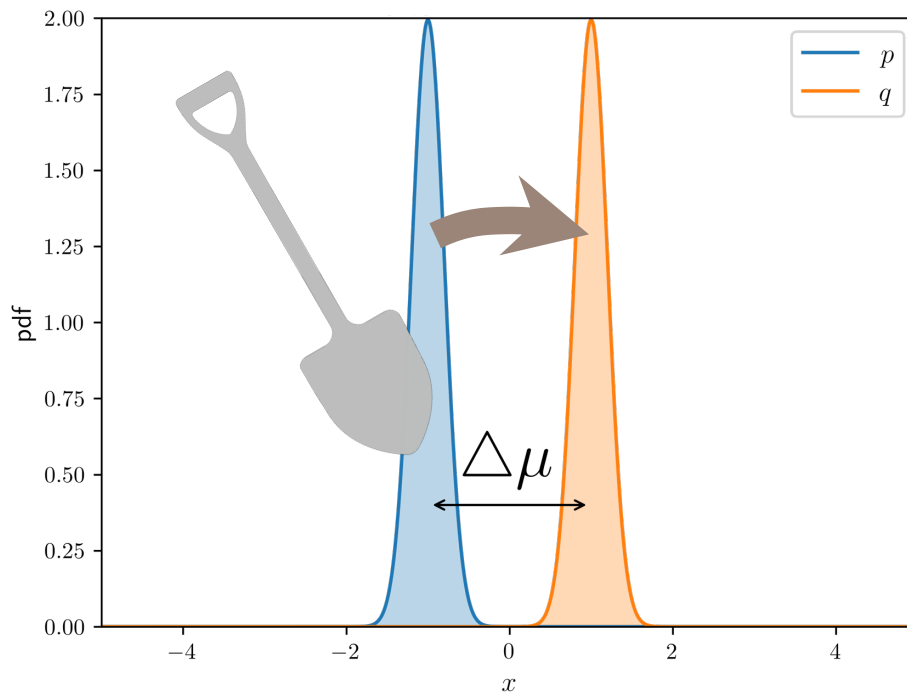
Wasserstein GAN

- Problem of D_{JS} : if p and q don't overlap, D_{JS} is a constant, i.e., no gradient
- D_{JS} is useful only if p and q are close



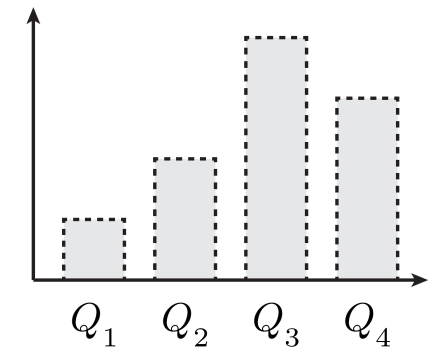
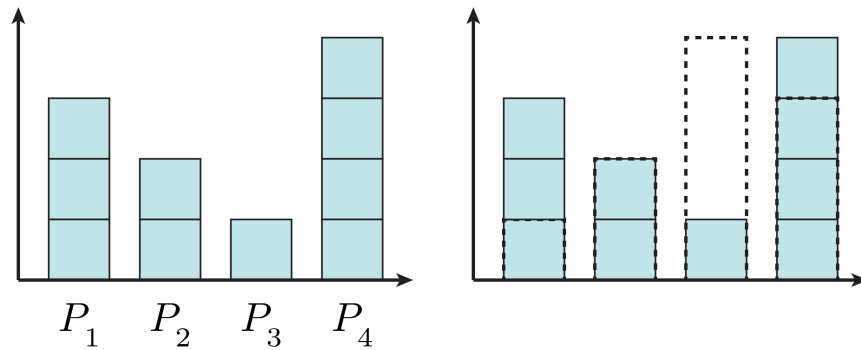
Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
 - The minimum amount of work required to change one distribution into the other



Wasserstein GAN

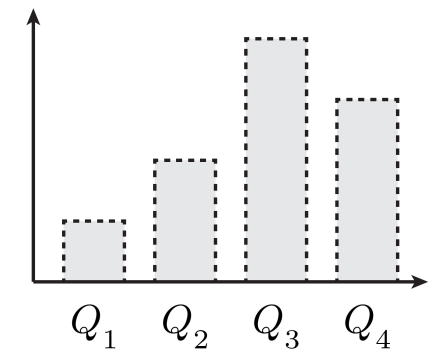
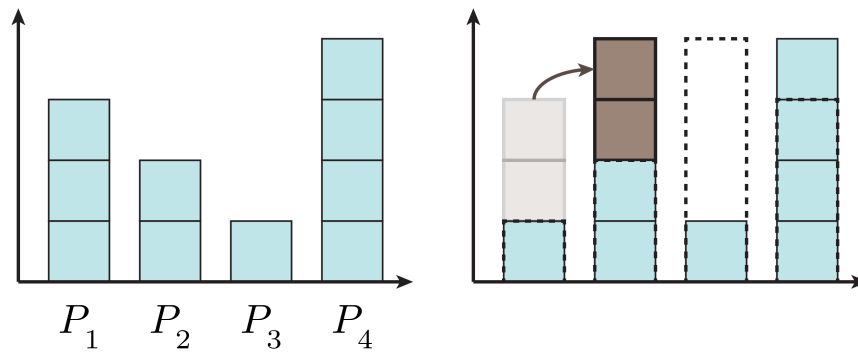
- Wasserstein Distance / Earth Mover's Distance:
 - The minimum amount of work required to change one distribution into the other



Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
 - The minimum amount of work required to change one distribution into the other

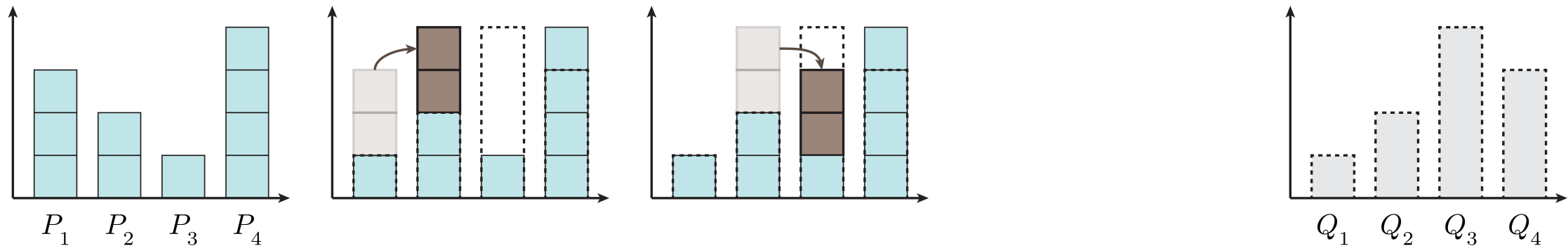
2 shovelfuls



Wasserstein GAN

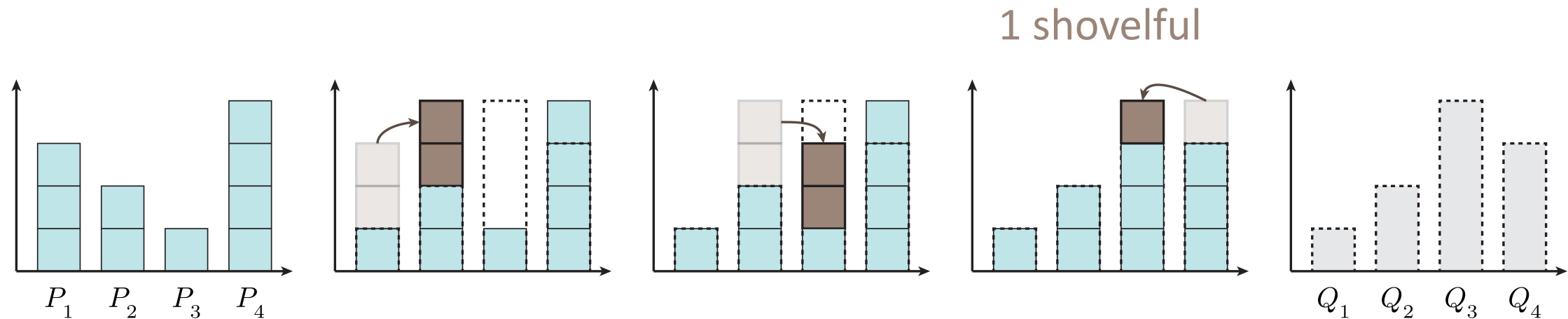
- Wasserstein Distance / Earth Mover's Distance:
 - The minimum amount of work required to change one distribution into the other

2 shovelfuls



Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
 - The minimum amount of work required to change one distribution into the other



Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
 - The minimum amount of work required to change one distribution into the other

- 1-Wasserstein Distance (1-d, discrete)

l_1 -norm

$$W_1(P, Q) = \sum_i |\text{cdf}_P(i) - \text{cdf}_Q(i)|$$

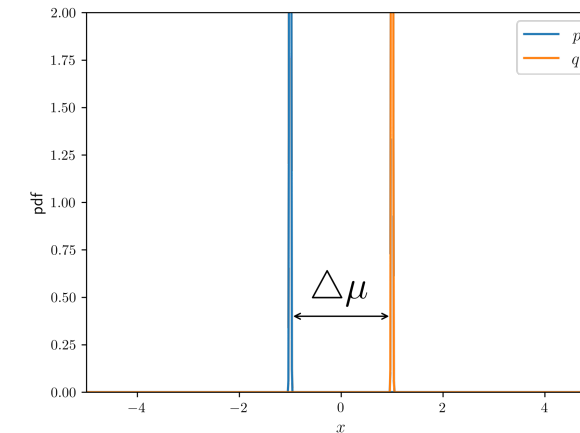
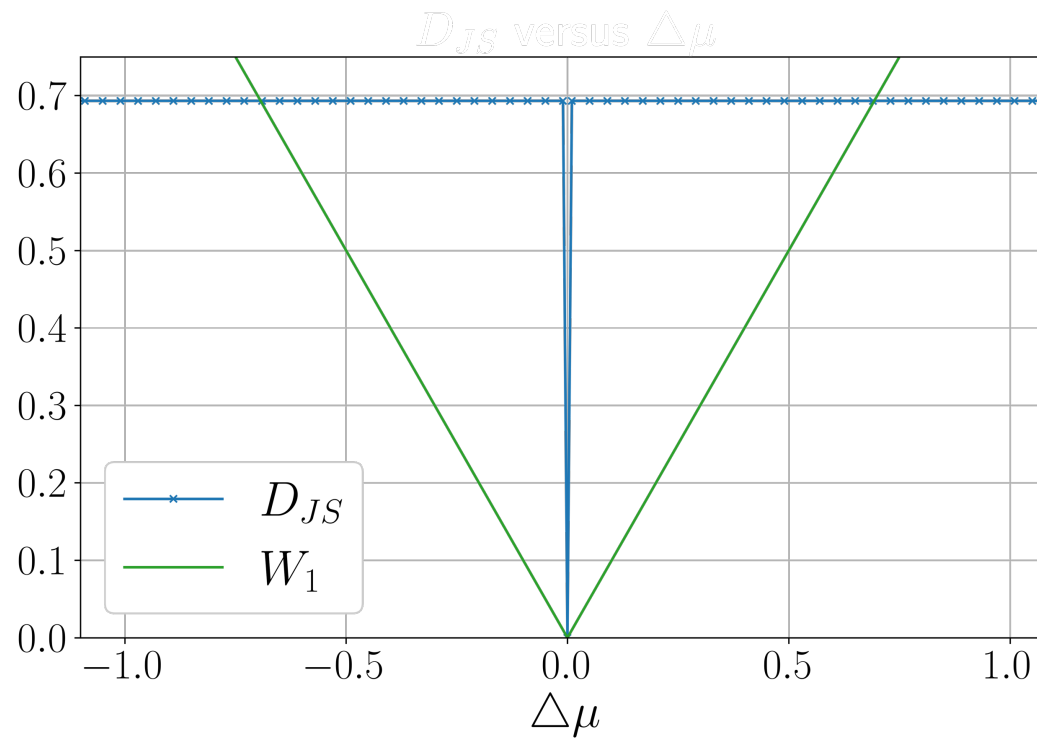
cdf not pdf

- 1-Wasserstein Distance (1-d, continuous)

$$W_1(p, q) = \int_x |\text{cdf}_p(x) - \text{cdf}_q(x)| dx$$

Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
 - The minimum amount of work required to change one distribution into the other



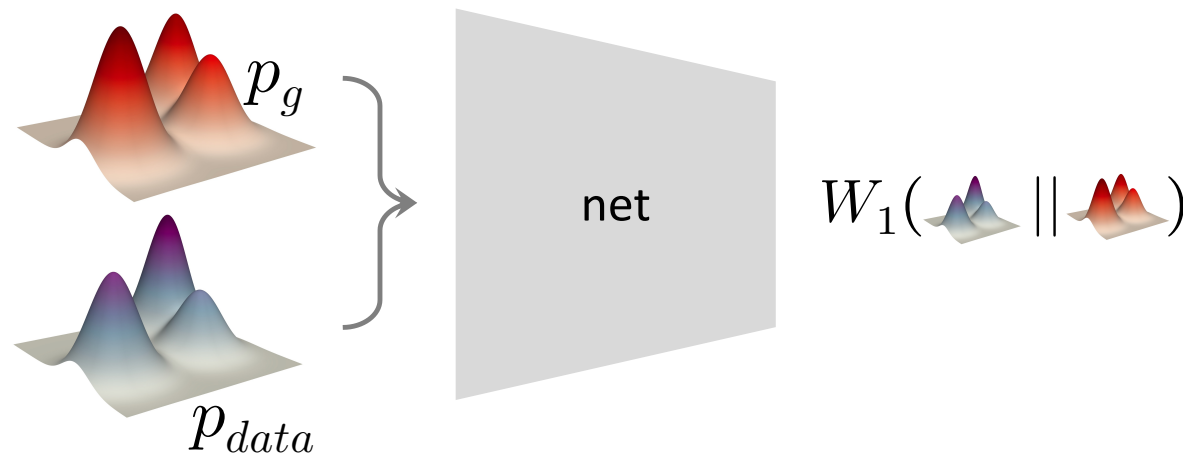
Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
 - Intractable to compute as a GAN learning objective

- 1-Wasserstein Distance (high-dim, continuous)

$$W_1(p, q) = \inf_{\gamma \in \Pi(p, q)} \mathbb{E}_{(x, y) \sim \gamma} [||x - y||]$$

- all joint distributions $\gamma(x, y)$
whose marginals are p and q



Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
 - Intractable to compute as a GAN learning objective

- 1-Wasserstein Distance (high-dim, continuous)

$$W_1(p, q) = \inf_{\gamma \in \Pi(p, q)} \mathbb{E}_{(x, y) \sim \gamma} [|x - y|]$$

- all joint distributions $\gamma(x, y)$
whose marginals are p and q



Kantorovich-Rubinstein duality:

$$W_1(p, q) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p}[f(x)] - \mathbb{E}_{x \sim q}[f(x)]$$

- all 1-Lipschitz functions

Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
 - Intractable to compute as a GAN learning objective

$$W_1(p, q) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p}[f(x)] - \mathbb{E}_{x \sim q}[f(x)]$$

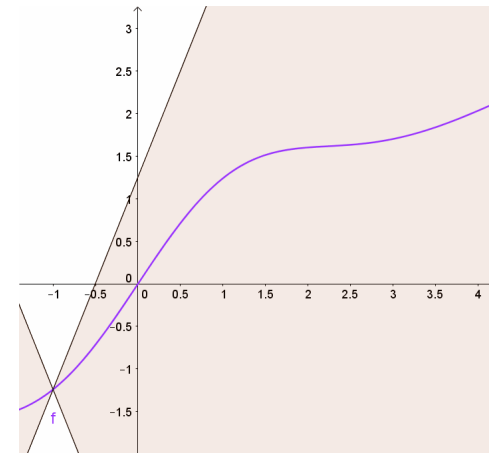
• all 1-Lipschitz functions

K -Lipschitz continuity:

$$|f(x) - f(y)| \leq K|x - y|, \quad \forall x, y$$

gradient is bounded:

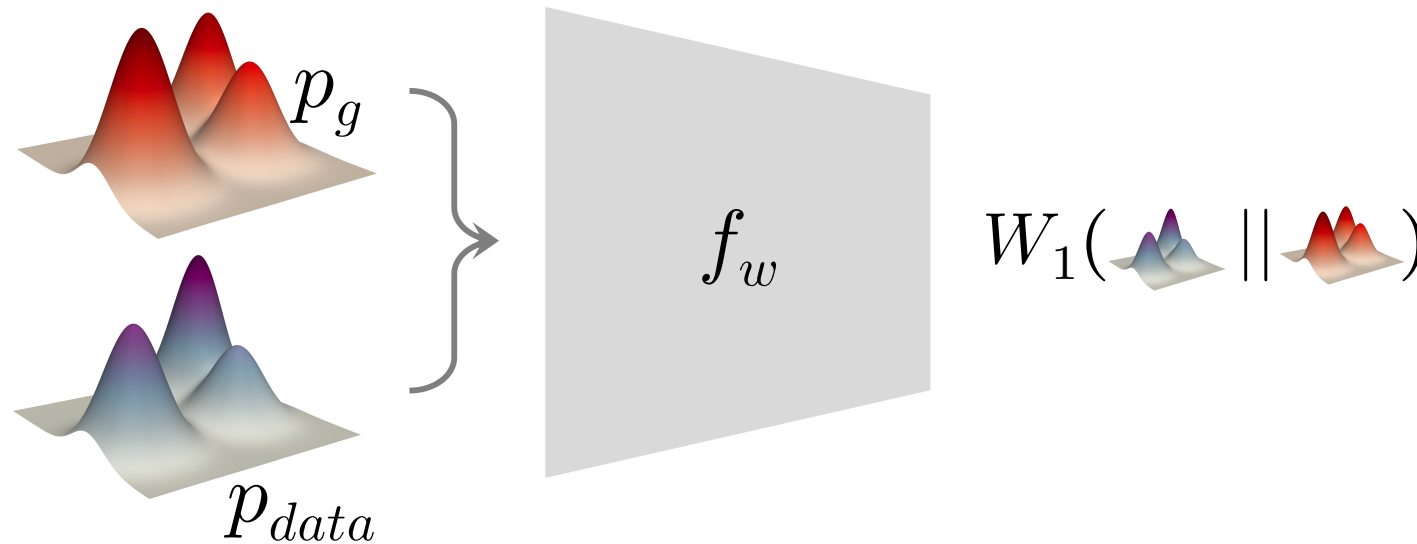
$$\frac{|f(x) - f(y)|}{|x - y|} \leq K$$



Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
- **Final Objective:**

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim p_{\text{data}}} [f_w(x)] - \mathbb{E}_{x \sim p_g} [f_w(x)]$$

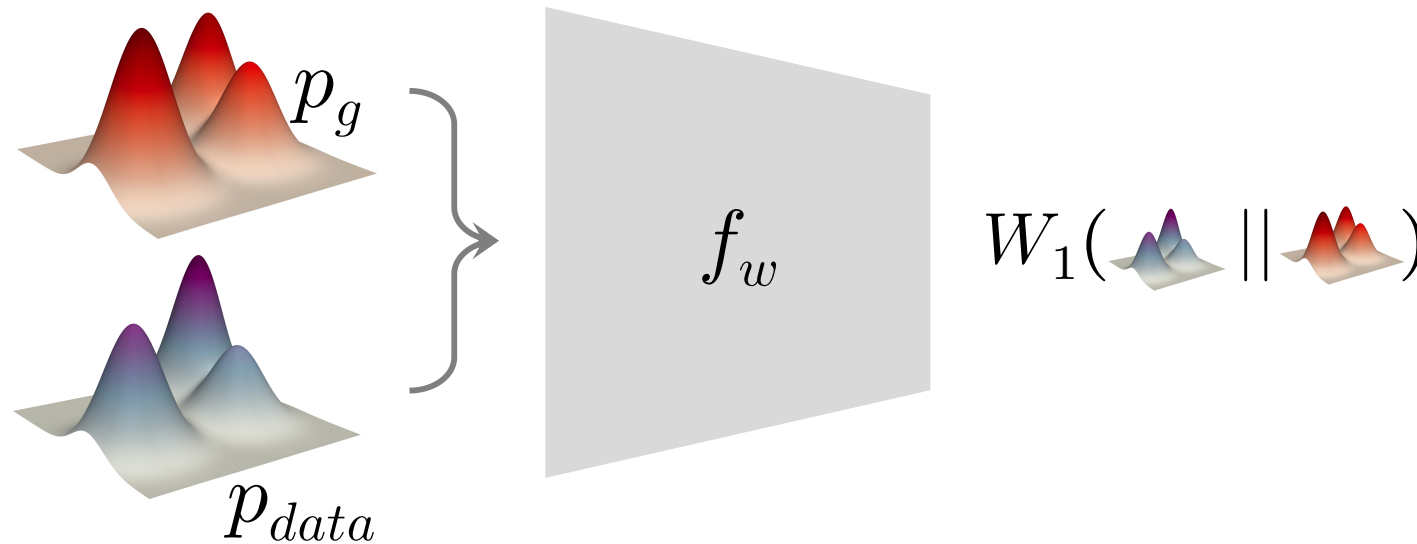


Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
- **Final Objective:**

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim p_{\text{data}}} [f_w(x)] - \mathbb{E}_{x \sim p_g} [f_w(x)]$$

- weights are bounded: in practice, clipped $[-0.01, 0.01]$



Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
- **Final Objective:**

- W-GAN's objective function:

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim p_{\text{data}}} [f_w(x)] - \mathbb{E}_{x \sim p_g} [f_w(x)]$$

“art critic”

- value/merit/quality/...
- direction to improve (gradients)

- original GAN's objective function (D-step):

$$\max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{x \sim p_g} [\log(1 - D(x))]$$

“forgery expert”

- real/fake

Wasserstein GAN

- Wasserstein Distance / Earth Mover's Distance:
- **Final Objective:**

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim p_{\text{data}}} [f_w(x)] - \mathbb{E}_{x \sim p_g} [f_w(x)]$$

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α , the learning rate. c , the clipping parameter. m , the batch size. n_{critic} , the number of iterations of the critic per generator iteration.

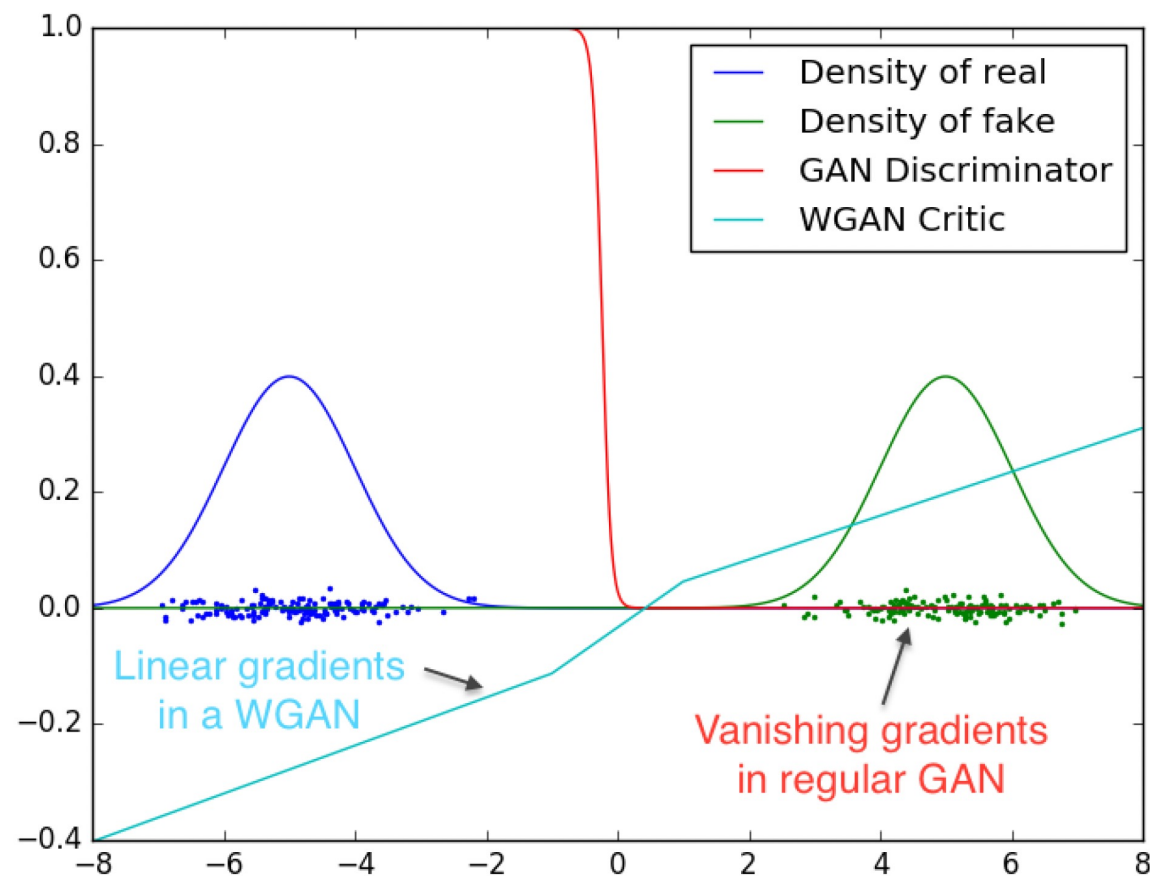
Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w [\frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

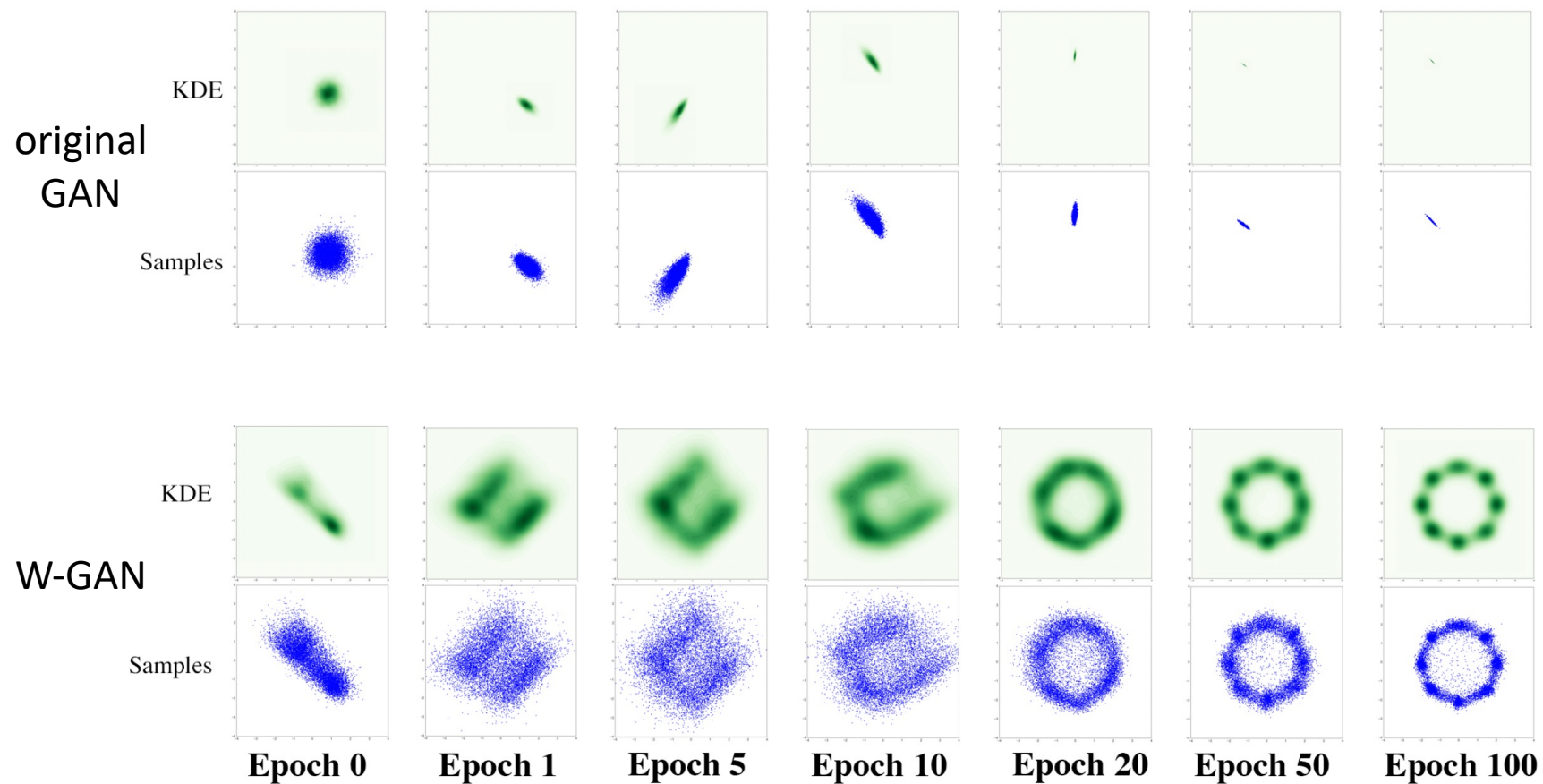
D-step

G-step

Wasserstein GAN



Wasserstein GAN



Wasserstein GAN

For mathematicians:

- Wasserstein distance, instead of JS divergence

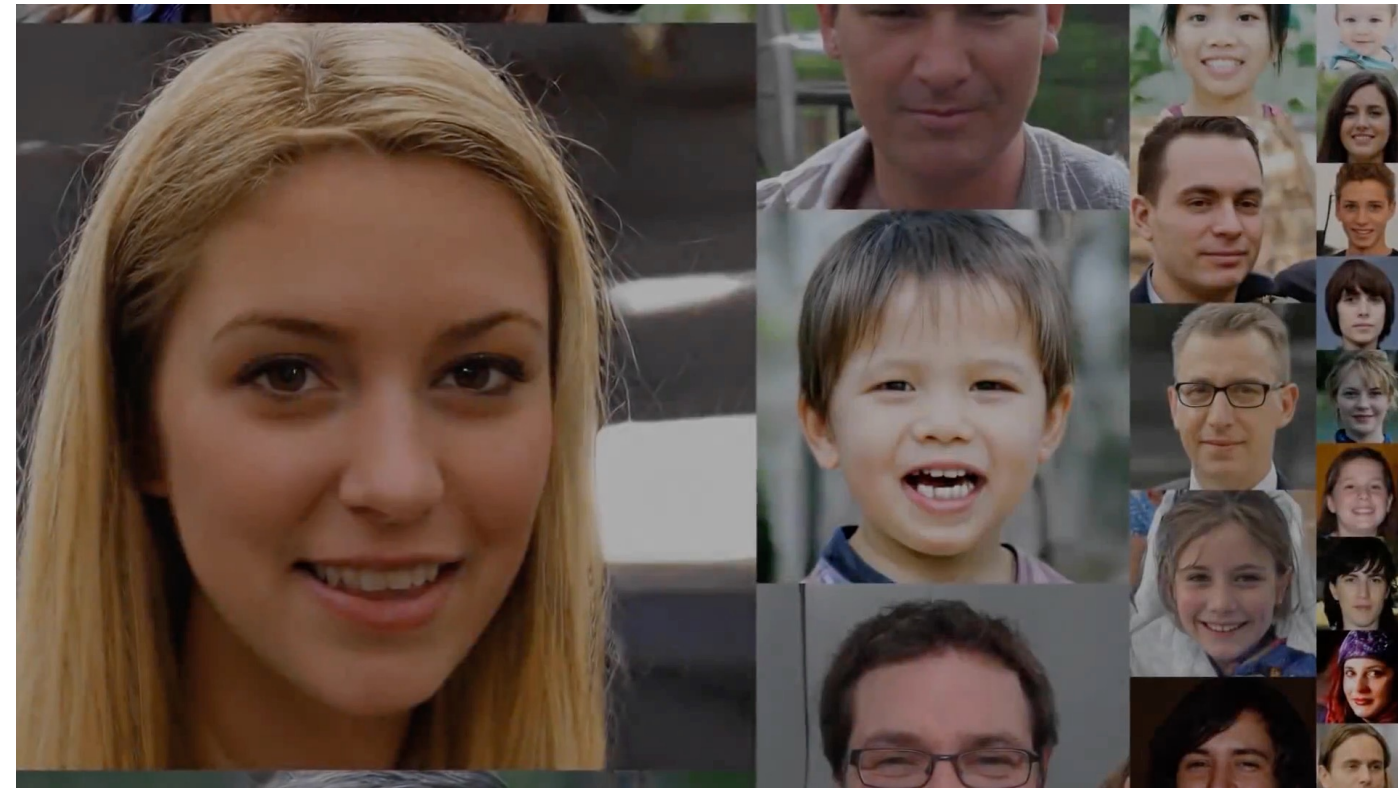
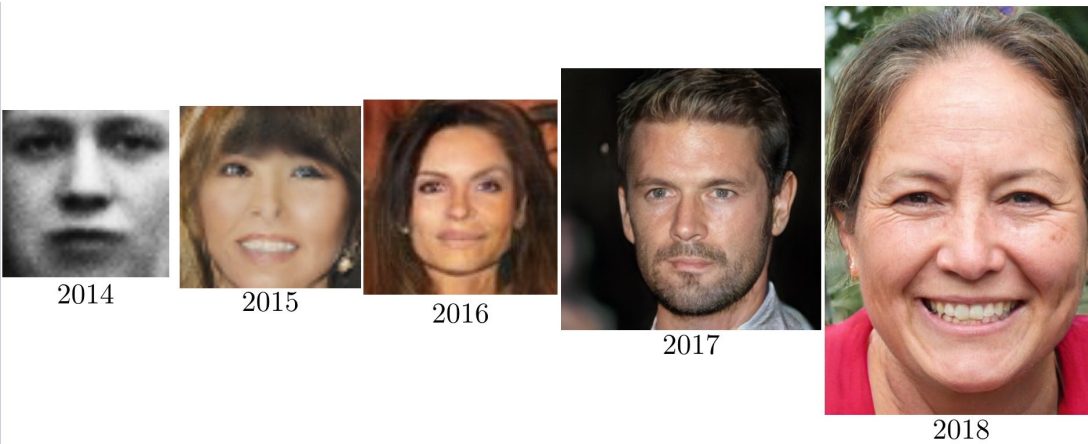
For engineers:

- remove logarithms **Wasserstein distance**
- clip weights **Lipschitz continuity**

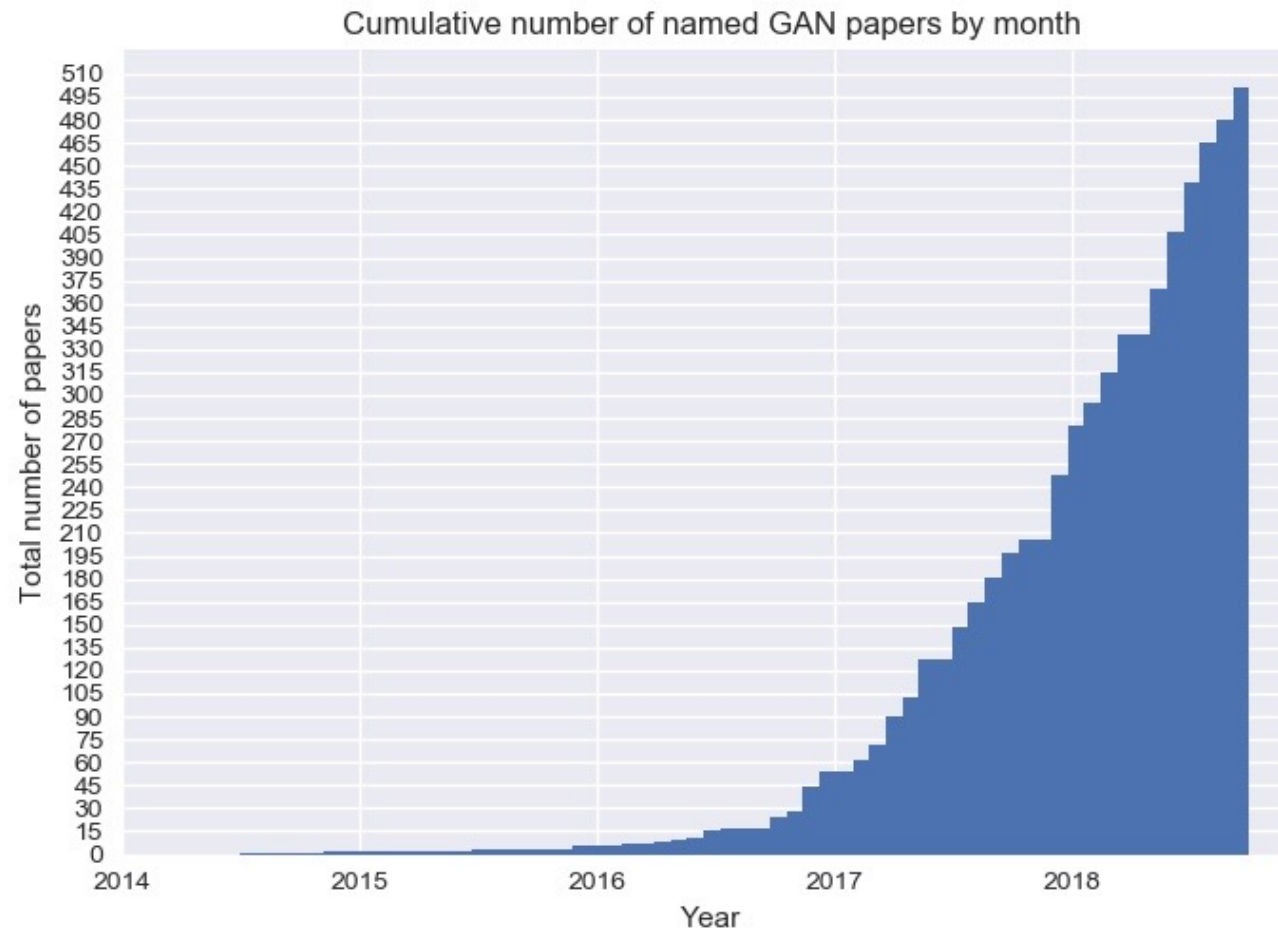
For laymen:

- art critic instead of a forgery expert
gradients

GAN Training is hard, yet **results are very promising**



GAN Training is hard, yet **results are very promising**



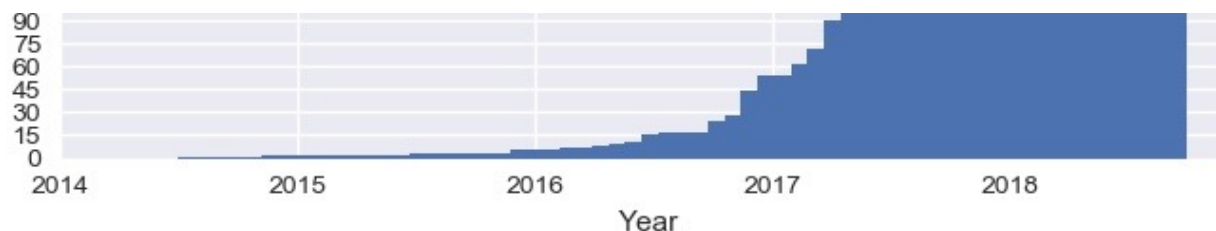
GAN Training is hard, yet **results are very promising**

Cumulative number of named GAN papers by month

Diffusion Models Beat GANs on Image Synthesis

Prafulla Dhariwal*
OpenAI
prafulla@openai.com

Alex Nichol*
OpenAI
alex@openai.com

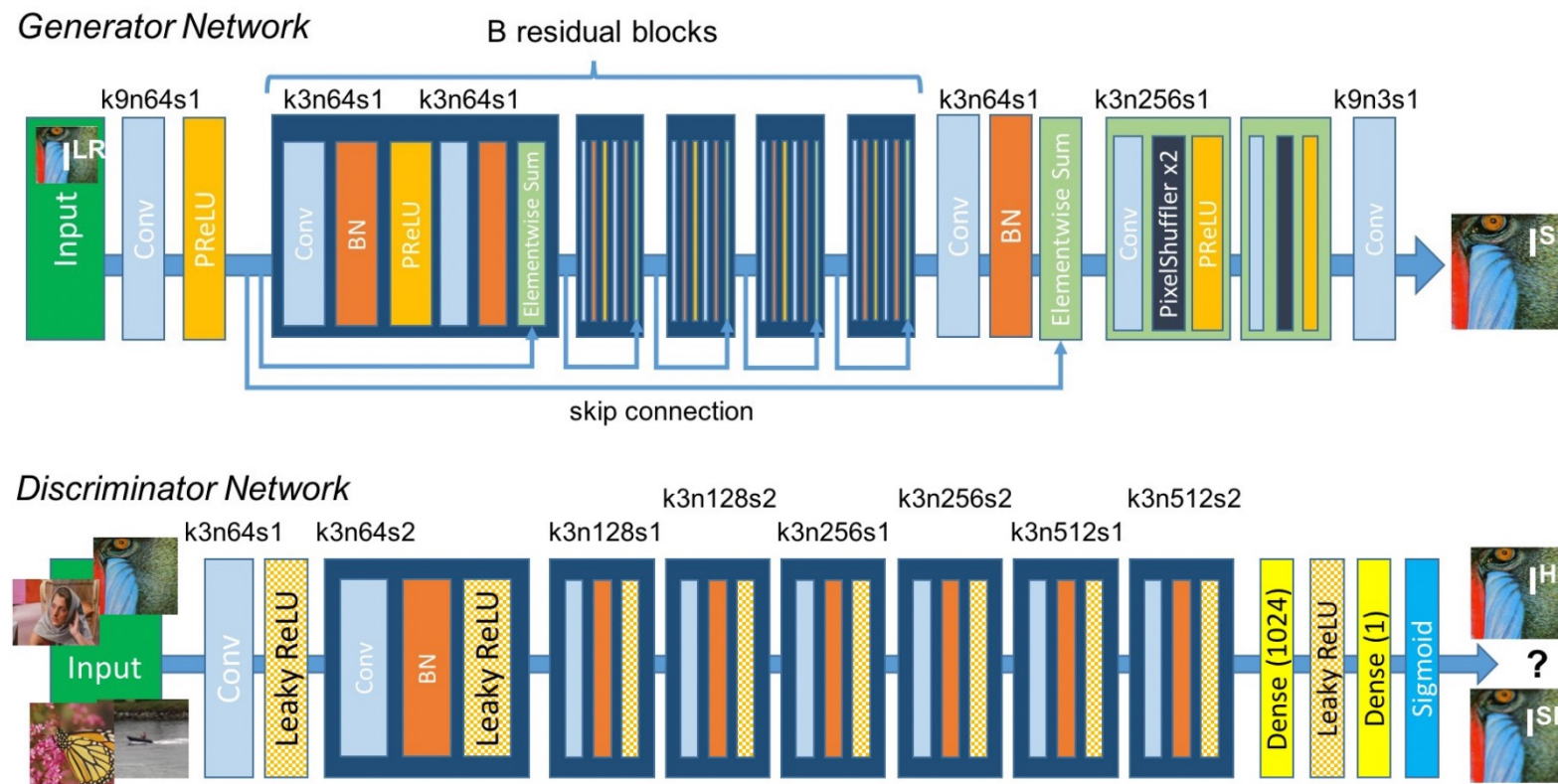


Still useful as an additional **adversary loss function**

- GAN essentially defines **an adversarial loss**
- Input to the discriminator can be **from another source**
- adversarial loss encourages output to **look more “realistic”**

Still useful as an additional **adversary loss function**

- Super-resolution



Still useful as an additional **adversary loss function**

- Super-resolution

- **better PSNR**

- worse PSNR, but better visual quality

original



bicubic
(21.59dB/0.6423)



SRResNet
(23.44dB/0.7777)



SRGAN
(20.34dB/0.6562)



Still useful as an additional **adversary loss function**

- Super-resolution

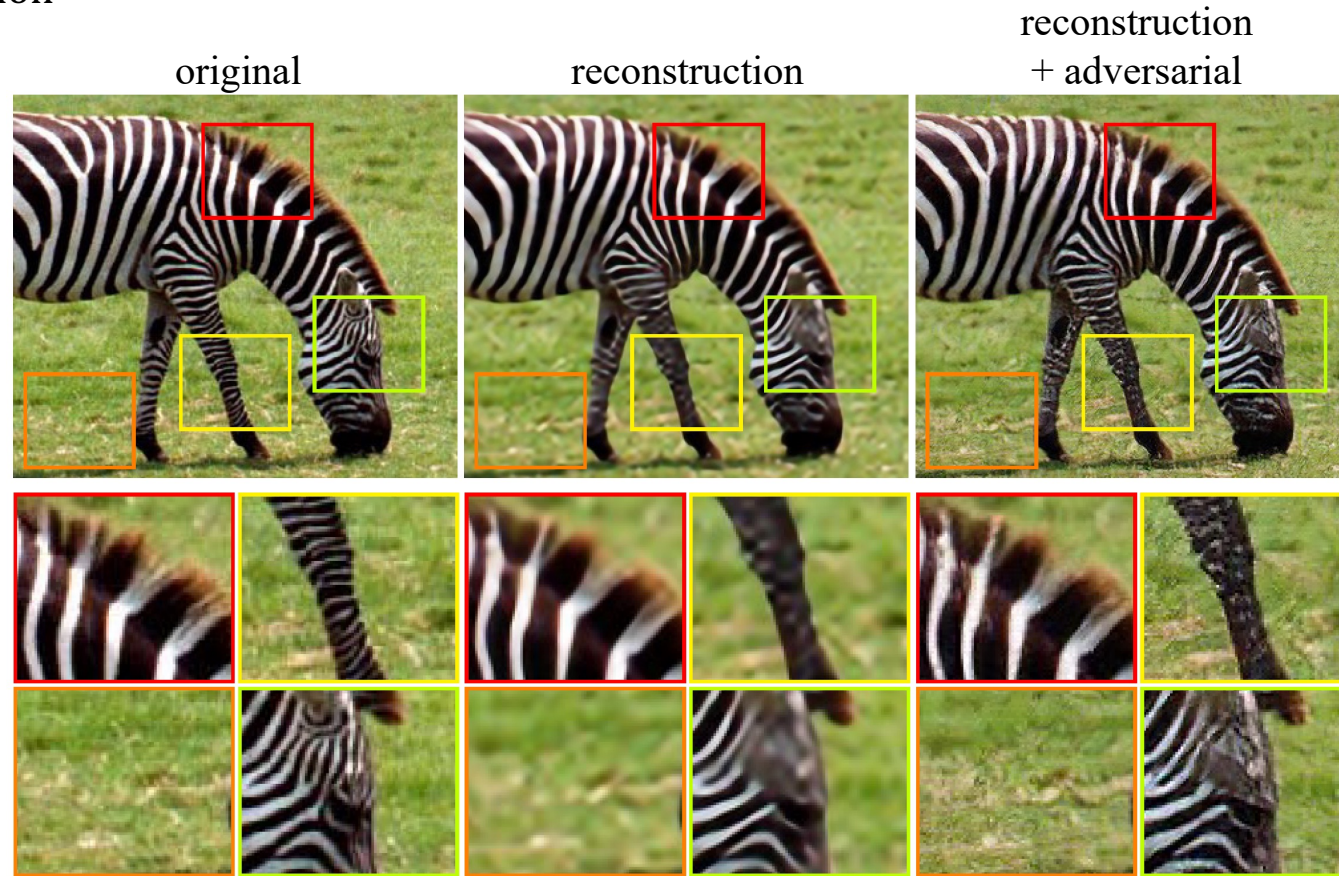


Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network, CVPR 2017

Still useful as an additional **adversary loss function**

- image translation

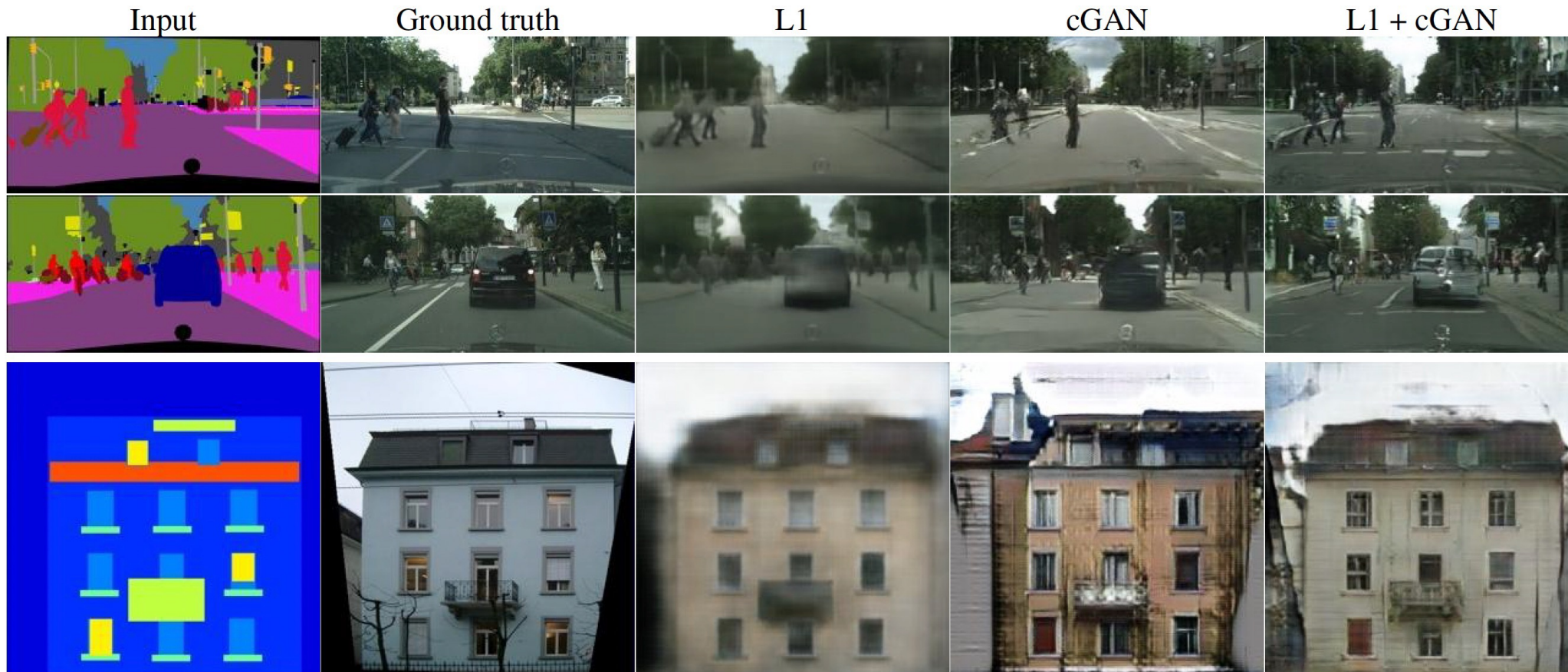
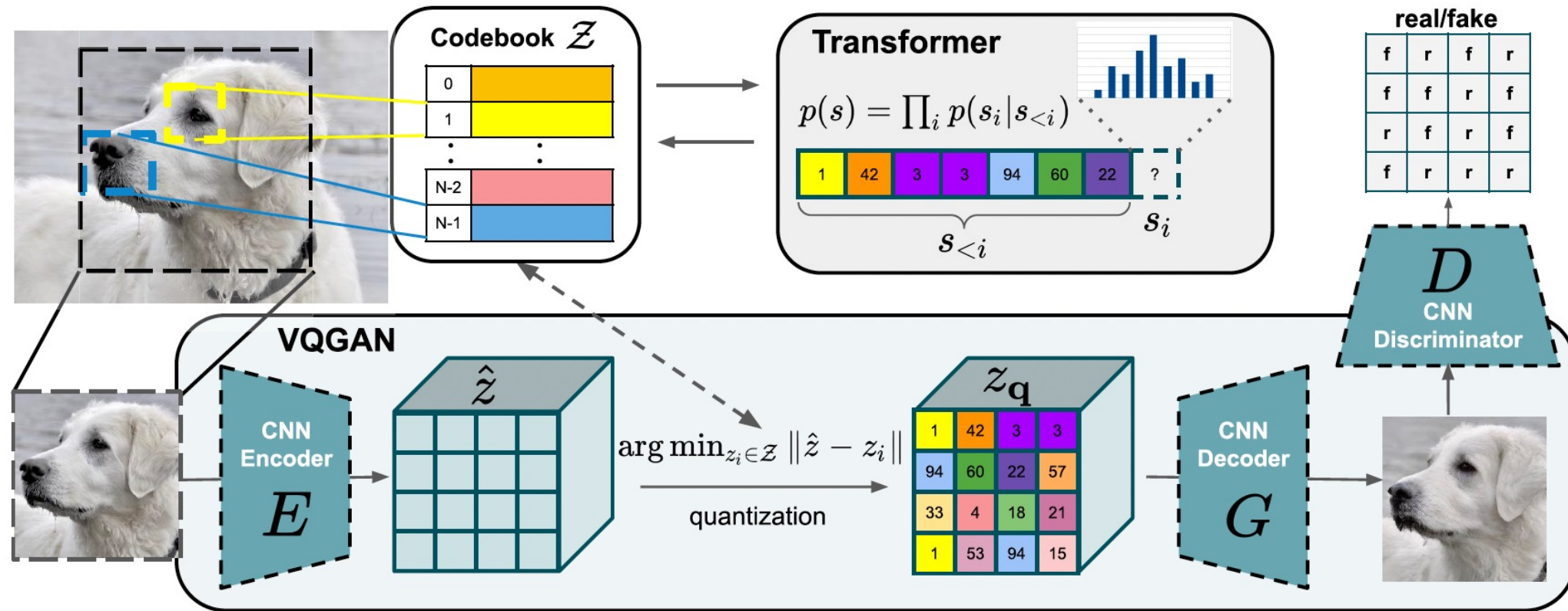


Image-to-Image Translation with Conditional Adversarial Networks, CVPR 2017

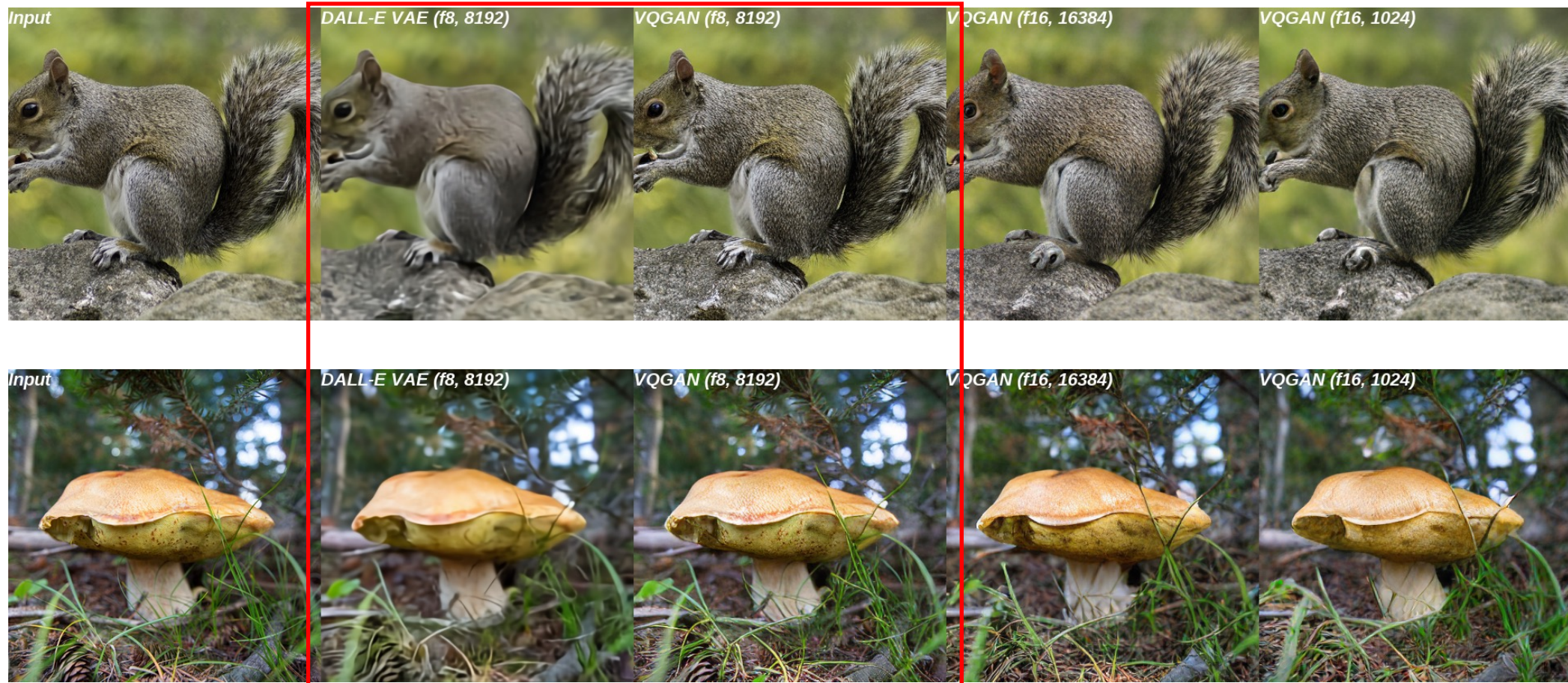
Still useful as an additional **adversary loss function**

- VQ-VAE to VQ-GAN



Still useful as an additional **adversary loss function**

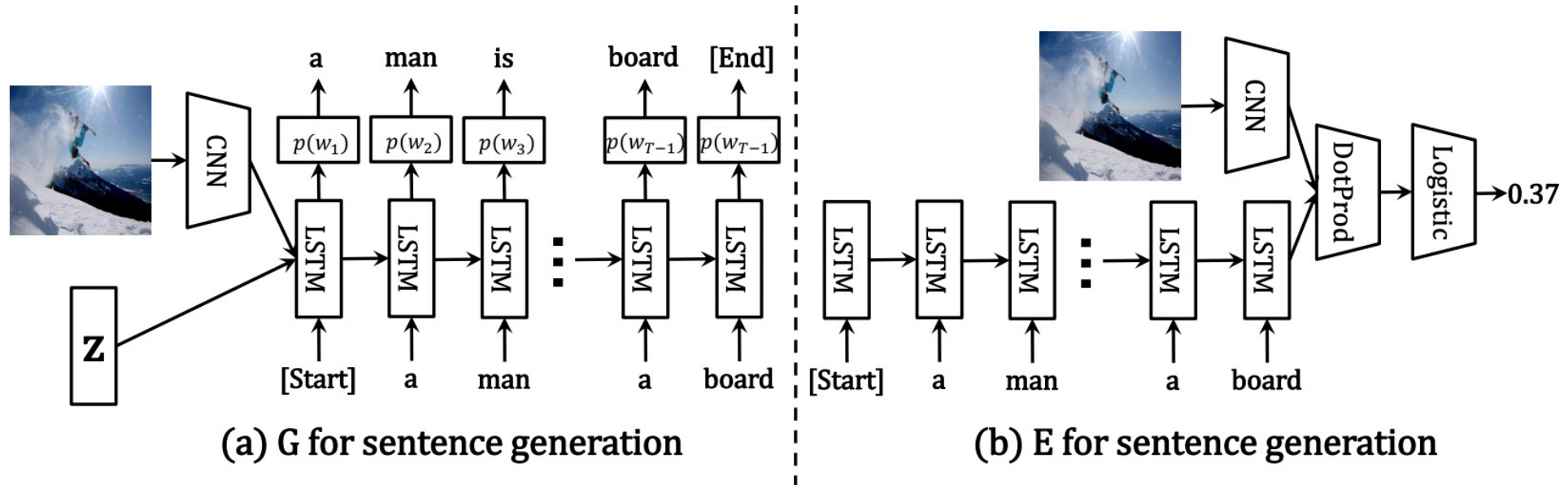
- VQ-VAE to VQ-GAN



Taming Transformers for High-Resolution Image Synthesis, CVPR 2021

Still useful as an additional **adversary loss function**

- Image-to-text / image captioning



Still useful as an additional **adversary loss function**

- Image-to-text / image captioning

				
G-MLE	a man flying through the air while riding a snowboard	a man flying through the air while riding a snowboard	a man flying through the air while riding a snowboard	a man flying through the air while riding a skateboard
G-GAN	a man on a skateboard in a snowy park	a man skiing down the slope near a mountain	a man performing a grind trick on a skateboard ramp	a man with stunts on his skis in the snow
				
G-MLE	a group of people standing around a boat	a group of people sitting around a table	a group of people sitting at a table	a group of people sitting around a living room
G-GAN	the bench is sitting on the ground by the water	a group of people watching each other	a table with a lot of stuff on it	furnished living room with furniture and built area

Towards Diverse and Natural Image Descriptions via a Conditional GAN, ICCV 2017

Still useful as an additional **adversary loss function**

- Generative adversarial imitation learning

